

MIPSolvers 原生 MILP 分支切割引擎

数学模型与严苛的数据结构与算法效率审计

代码级审计报告（静态分析，所有论断附 file:line 引用）

2026 年 8 月 1 日

摘要

本报告直接依据源代码，记录三方面的内容：(i) MIPSolvers 仓库原生 MILP 求解器所接受的数学模型，以及其分支切割机器（预求解、LP 松弛处理、割平面、分支、节点选择、原始启发式、域传播、冲突学习与并行化）的完整数学内容；(ii) 以国际先进 MILP 求解器工程水准（SCIP、HiGHS、Gurobi）为基准，对其数据结构与算法效率进行的刻意严苛的审计；(iii) 对“占位符层”进行的系统性、可复现的普查——即那些被声明、被宣传、甚至被执行，却不产生任何计算的机器。每项技术论断均附仓库内的 `file:line` 引用；行号对应报告日期当日的工作区状态。本审计为静态审计：未做运行时性能剖析，复杂度陈述均源自代码本身。凡代码意图不明之处，文中明确指出，不做臆测。

目录

1 范围、方法与文件清单	2
1.1 范围	2
1.2 方法	2
1.3 文件清单与活跃性	2
2 数学模型	3
2.1 接受的问题类	3
2.2 整数性的双重指定	4
2.3 松弛与定界的形式化	4
3 架构与执行路径拓扑	5
3.1 遗留单体是唯一的活跃路径	5
3.2 默认路径：并行，基于 vendored HiGHS LP 内核	5
3.3 顶层流水线	6
4 预求解	6
4.1 接线：PaPILO 优先，原生预求解作为后备	6
4.2 原生预求解：表示与约减	7
4.3 经核实缺失的标准约减	8

目录	2
5 LP 松弛层	8
5.1 三层嵌套表示，逐节点拷贝流转	8
5.2 界变更：SF 层是增量的，在求解器边界被丢弃	9
5.3 逐节点求解：全新 HiGHS，仅状态的热启动	9
5.4 向 LP 添加割与重新求解	10
5.5 容差、缩放与迭代上限	10
5.6 失败处理与回退层级	11
6 割生成	11
6.1 派发与 All 套件	11
6.2 变换单纯形表 cMIR (“Gomory” 路径)	12
6.3 路径聚合、路径混合与模 k 割	12
6.4 模型行割平面族	13
6.5 团割与持久团表	14
6.6 CGLP 提升-投影	14
6.7 割选择与过滤	14
6.8 割池机制	15
6.9 根节点与树内割策略	15
7 分支、节点选择与原始启发式	16
7.1 伪成本模型与分支规则	16
7.2 节点选择	17
7.3 原始启发式	18
8 域传播、冲突学习与目标传播	19
8.1 BCDomain: 增量传播器	19
8.2 带理由跟踪的逐节点传播	19
8.3 冲突池：没有监视文字	19
8.4 蕴含图与变量界表	20
8.5 目标传播	20
9 并行化	20
9.1 设计	20
10 终止与间隙处理	22
11 综合严苛审计	23
11.1 严重度 1 ——与最先进水平的结构性性能差距	23
11.2 严重度 2 ——热路径上的算法性浪费	24
11.3 严重度 3 ——正确性边缘问题与有效性缺口	25
11.4 严重度 4 ——如今已损耗性能或信任的工程债	25
11.5 做得好的地方（为平衡起见）	26

目录	3
12 占位符与虚假复杂性普查	27
12.1 普查方法（可复现）	27
12.2 定量结果	27
12.3 占位符层的定性分类	28
12.4 占位符层为何重要	29
13 优先级排序的建议	29
A 默认参数表（经代码核实）	30
B 记号	32
C 核实声明	32

1 范围、方法与文件清单

1.1 范围

本次审计的组件是原生 *MILP* 分支切割 (*B&C*) 引擎:

```
src/engine/solver/native/milp/bc/    (实现)
include/mipsolvers/engine/bc/, include/mipsolvers/engine/detail/, include/
mipsolvers/engine/solver/native/milp/bc/    (头文件)
```

以及它所驱动的 LP 内核接口 (`src/engine/kernel/lp_kernel/`)。外部求解器适配器 (HiGHS-MIP、SCIP、Gurobi) 不在审计范围内, 唯原生引擎委托给内嵌 (vendored) HiGHS LP 内核之处除外。

1.2 方法

对代码进行了八个子系统轮次的审查 (公共 API/选项、核心数据结构、主 B&C 引擎、割生成、预求解/传播、搜索/分支/并行、LP 松弛层, 以及工具层)。随后, 本报告中所有关键论断均通过直接阅读所引用的代码行逐一复核。另有一项针对选项/统计表面 (按声明字段统计的成员访问读取次数) 以及隐藏配置通道的独立定量普查, 见 §12.1。报告在关键之处引用精确常量与简短代码摘录。本报告区分三类陈述: **(V)** 经直接阅读所引代码验证 (占绝大多数); **(I)** 由代码结构推断, 并明示推断依据; **(U)** 代码本身不清晰/含糊——显式标注。

1.3 文件清单与活跃性

表 1 列出了主要文件。一个塑造全篇报告的事实: 生产环境引擎是一个单一的 *legacy* 翻译单元, 而“现代”的 `search/`、`parallel/` 和 `root/` 模块虽被编译 (`CMakeLists.txt:204--209`), 但在其自身文件之外 **没有任何调用点** (已在 `src/` 和 `tests/` 范围内经 `grep` 验证)。

路径简写。 全文中, `bac.cpp` = `src/engine/solver/native/milp/bc/legacy/branch_and_cut.cpp`; `bc_cuts.cpp`、`bc_utils.cpp`、`bc_branching.cpp`、`bc_parallel.cpp`、`bc_relaxation.cpp`、`bc_legacy_helpers.cpp` 为 `legacy/` 中的同级文件; `options.hpp` = `include/mipsolvers/engine/bc/options.hpp`; `bc_types.hpp`、`bc_pools.hpp`、`bc_domain.hpp`、`bc_threading.hpp`、`bc_utils.hpp` = `include/mipsolvers/engine/detail/` 中的对应文件; `dual_simplex.hpp`、`cpp`、`dual_simplex_api.cpp` = `include/mipsolvers/engine/kernel/lp_kernel/` 与 `src/engine/kernel/lp_kernel/` 中的同名文件; `problem_types.hpp` = `include/mipsolvers/engine/problem_types.hpp`。

表 1: 原生 MILP B&C 引擎的主要文件 (大小以行数计)。

文件	行数	职责
legacy/branch_and_cut.cpp	36 921	整个存活引擎
legacy/bc_cuts.cpp	8 364	割分离器
legacy/bc_utils.cpp	6 352	传播、队列、对偶证明
legacy/bc_relaxation.cpp	3 288	根节点 LP、HiGHS 互操作
milp_presolve.cpp	1 669	原生预求解
legacy/bc_parallel.cpp	1 644	并行探索器 (存活)
legacy/bc_implied_bounds.cpp	1 311	隐含整数/VB/团遗留构件
legacy/bc_branching.cpp	1 089	分支规则
legacy/bc_clique_table.cpp	999	持久化团表
legacy/bc_cglp.cpp + bc_cglp_model.cpp	677	CGLP 提升-投影
search/ (2 个驱动文件)	696	死代码
parallel/ (2 个驱动文件)	452	死代码
root/root_solve.cpp	284	死代码
detail/bc_utils.hpp	3 270	节点队列、蕴含图 (内联)
detail/bc_objective_propagation.hpp	2 368	目标传播 (仅头文件)
detail/bc_threading.hpp	1 266	线程安全队列、共享池
detail/bc_pools.hpp	1 070	割/冲突/解池
detail/bc_types.hpp	884	Node, PoolCut, PseudoCost, ...
bc/options.hpp	881	BCOptions (~230 个公共字段)
detail/bc_domain.hpp	512	BCDomain 增量传播

2 数学模型

2.1 接受的问题类

原生求解器恰好接受如下混合整数线性规划 (**V**: LPModel 见 problem_types.hpp:127--146, MIPModel 见 problem_types.hpp:269--277):

$$\begin{aligned}
 \min_{x \in \mathbb{R}^n} \quad & c^\top x \\
 \text{s.t.} \quad & \ell^{\text{row}} \leq Ax \leq b, \quad A \in \mathbb{R}^{m_1 \times n}, \ell^{\text{row}} \in (\mathbb{R} \cup \{-\infty\})^{m_1}, b \in (\mathbb{R} \cup \{+\infty\})^{m_1}, \\
 & A^{\text{eq}}x = b^{\text{eq}}, \quad A^{\text{eq}} \in \mathbb{R}^{m_2 \times n}, \\
 & \ell_j \leq x_j \leq u_j, \quad j = 1, \dots, n, \ell_j \in \mathbb{R} \cup \{-1\text{e}20\}, u_j \in \mathbb{R} \cup \{+1\text{e}20\}, \\
 & x_j \in \mathbb{Z}, \quad j \in \mathcal{I} \subseteq \{1, \dots, n\}, \\
 & x_j \in \{0, 1\}, \quad j \in \mathcal{B} \subseteq \{1, \dots, n\}.
 \end{aligned} \tag{1}$$

结构性事实:

- **目标: 仅线性**, 方向为 Minimize/Maximize (problem_types.hpp:20, 128--129)。不存在二次目标, 且 LPModel 中没有目标常数/偏移字段——MPS 文件中的常数必须在模型结构体之外另行跟踪。

- **约束：仅线性。**不等式行原生为 双侧 (*range*) 行: `row_lhs` 向量为空表示所有下侧均为 $-\infty$ (`problem_types.hpp:130--136`; 辅助函数 `lp_has_row_lhs/lp_row_lhs_or_neg_inf` 见:148–161)。等式单独存储 (`Aeq`、`beq`)。
- **变量类型**恰好为 `Continuous`、`Integer`、`Binary` (`problem_types.hpp:18`)。没有 `SOS`，没有半连续/半整数变量，没有指示约束（在全仓库范围内 `grep SOS`/半连续，仅命中内嵌 HiGHS 的 LP 文件读取器，从未命中原生模型或 B&C 头文件）。
- 事实上的无穷界为 $\pm 10^{20}$ (`VariableMeta{lb{-1e20}, ub{1e20}}`), `problem_types.hpp:105--110`); `kInf = 1020` 同时也是预求解的无穷大 (`milp_presolve.cpp:32`)。
- 一个领域特定的 `UCGenHint` (机组组合元数据: 机组分段、最小开/停机时间、爬坡、PTDF 行、储能循环、十个 `certifies_*` 标志) 被嵌入通用的 `MIPModel` (`problem_types.hpp:279--406`)，且仅在设置 `enable_domain_heuristics` 时才会被读取。

2.2 整数性的双重指定

整数性被指定了两次：逐变量的 `VariableMeta.type` 与索引向量 `integer_idx/binary_idx`。二者由 `normalize_declared_integrality` (`bc_legacy_helpers.cpp:156--189`) 合并，该例程会静默修改变量类型，并强制将二进制变量的界钳制到 $[0, 1]$ 。一般整数变量只得到类型赋值，但没有界健全性检查：自由整数变量 ($\ell_j = -10^{20}, u_j = +10^{20}$) 不受任何质疑地进入分支定界——SCIP、HiGHS 和 Gurobi 都会拒绝或特殊处理无界整数变量，因为分支要求有限的子域。同一索引同时出现在两个列表中仅在引擎路径上才报错 (`src/engine/util/problem_validation.cpp:262--270`, 由 `src/engine/api/solver.cpp` 调用); 直接的 `solve_milp_bc` 入口 (`src/engine/solver/native/milp/bc/api.cpp:7--9`) 不执行此类验证。

2.3 松弛与定界的形式化

在内部，LP 松弛 (1) (去掉整数性) 在每次求解时被一次性转换为带平移、带符号的标准形 (`StandardFormLP`, `dual_simplex.hpp:273--313`):

$$\min \bar{c}^\top \bar{x} \quad \text{s.t.} \quad \bar{A}\bar{x} = \bar{b}, \quad 0 \leq \bar{x} \leq \bar{u}, \quad (2)$$

其中 $\bar{x} = D_c^{-1}(x - \ell^{\text{shift}})$ ，此前经过 Ruiz 均衡化 ($D_r A D_c$ 、 $D_r b$ 、 $D_c c$ ，至多 10 轮， ≥ 2 轮后当最大偏差 $< 10^{-2}$ 时提前停止; `dual_simplex.hpp:361`, `dual_simplex_api.cpp:818`)，并含显式的松弛/剩余/人工列，且对每行施加 `row_sign` 翻转。该标准形将约束矩阵存储了两次，列主序与行主序各一份 (`dual_simplex.hpp:277--278`)。

对偶界来自各节点处的 LP 对偶；现任解截断值镜像自 HiGHS 的 `computeNewUpperLimit` (`bac.cpp:1221--1239`): 当目标尺度 s 为整数时， $U = (\lceil s \cdot z - \text{tol} \rceil - 1)/s + \text{tol}$ ，否则为 $\min(z - \text{tol}, \text{nextafter}(z, -\infty))$ 。不可行剪枝要求 Farkas 证书 (`bc_fallback.hpp:82--84`); 数值失败绝不允许剪枝 (`bc_fallback.hpp:232, 268--270`)——这是本代码库中真正健全的设计决策之一。

3 架构与执行路径拓扑

3.1 遗留单体是唯一的活跃路径

全部四个公共入口点都无条件转发至遗留核心 (V: `api.cpp:7--27`):

```
BCResult solve_milp_bc(const MIPModel& prob, const BCOptions& opt) {
    return detail::solve_milp_bc_legacy_core(prob, opt);    // api.cpp:7-9
}
```

`solve_milp_bc_legacy_core` (`bac.cpp:36634--36638`) 调用 `branch_and_cut_lp`, 后者是横跨第 1568–36629 行 ($\approx 35\,000$ 行) 的单个函数 (V: 定义位于 `bac.cpp:1568`, 闭合花括号位于 `bac.cpp:36629`, 匿名命名空间结束于:36632)。所有“子例程”都是其内部按引用捕获的 `lambda` (`process_single_child` 位于:30537, `adopt_tree_incumbent` 位于:23828, `build_node_lp_with_local_cuts` 位于:23663, 等等), 通过捕获共享数百个可变局部变量。

`search/`、`parallel/`、`root/` 模块——`SequentialSearchDriver`、`NodeEvaluator`、`ParallelSearchDriver`、`RootSolvePhase`——会被编译但不可达 (V: 对 `src/` 执行 `grep` 只能在其各自文件内部找到引用)。它们复制了活跃路径的逻辑, 且行为已经分化, 例如在 `include/mipsolvers/engine/detail/bc/branching/pseudocost.hpp:18` 中存在第二个相互冲突的 `detail::PseudoCost` 定义 (一个无人包含的孤儿头文件), 以及一套与生产实现 (`bac.cpp:35717--35720`) 不一致的角色分配方案 (`parallel/parallel_search.cpp:287--305`)。

3.2 默认路径: 并行, 基于 vendored HiGHS LP 内核

两个默认值决定了每次运行的形态:

- `num_threads{-1}` (`options.hpp:213`) 解析为 `std::thread::hardware_concurrency()`, 并以 $\geq \text{auto_parallel_min_threads}\{2\}$ (`bc_legacy_helpers.cpp:23--36`) 为下限截断; `par_mode = (resolved_threads > 1)` (`bac.cpp:14133`)。在任何多核机器上, 并行搜索树都是默认路径, 仅有两个使其串行化的例外: `parallel_serial_on_low_fractionality_root{true}` (`options.hpp:247`, `bac.cpp:14256--14261`) 和 `parallel_delay_until_incumbent{true}` (`options.hpp:243`), 后者在根节点启发式未能找到现任解时, 迫使整个剩余求解过程在单个线程上运行 (`bac.cpp:33828--33834`); 不存在树中段的串行 $\rightarrow$ 并行转换。
- `lp_kernel_backend{LpKernelBackend::HiGHS}` (`options.hpp:135--137`, 注释: “*HiGHS is the production default; ExperimentalNative is for kernel development only*”)。节点 LP 由 **vendored HiGHS 对偶单纯形**求解, 而非自研的原生对偶单纯形 (`dual_simplex.cpp:2054--2062` 路由至 `solve_standard_form_with_vendored_highs`; 原生内核仅在 `ExperimentalNative` 下才会被触及)。`bc/api.hpp:16--17` 中的公开文档注释仍声称原生对偶单纯形是默认值——过时文档。

注记 1 (命名陷阱). 引擎级默认 *MILP* 适配器列表为 `{"StrictHiGHS", "HiGHS", "NativeBranchAndCut"}` (`src/engine/strategy/dispatcher.cpp:78--79`)。尽管名字如此, *StrictHiGHS* 运行的是原生搜索树, *HiGHS* 仅充当 LP 预言机: 将整个 *MIP* 委托给 `Highs::run()` 受 `strict_highs_mip_contract` 与 `auto_highs_root_pipeline` (`bac.`

`cpp:1576--1579, 1703--1705`) 门控, 而 `strict_highs_mip_contract` 默认为 `false` (`options.hpp:141`), 且不会被 `make_strict_highs_production_options` (`src/engine/solver/native/native_adapters.cpp:395--420`) 设置。

3.3 顶层流水线

在 `branch_and_cut_lp` 内部, 各阶段按以下顺序执行 (均位于 `bac.cpp`; V 依据逐行阅读):

1. 选项/契约设置 (:1568–1600), 包括 `apply_bc_strict_highs_contract`;
2. PaPILO 预求解 (:2967; `use_papilo_presolve{true}`, `options.hpp:162`), 随后仅当 *PaPILO* 未缩减模型时才运行原生 `MILPPresolve` (:3015–3025);
3. 来自原始模型的隐含整数/变量界附属状态 (:2942–2955), UC 提示重映射 (:3440);
4. 探测/强分支预算计算 (:3632–3647): 若 $n \geq 10^5$ 或 $m > \text{large_problem_threshold}$ 则 `probe_reliability`= 0, 若 $m > 12\,000$ 则为 1, 否则为 `probe_reliability{2}`;
5. 根节点 LP 求解 (:5047);
6. 根节点域不动点机制 (~:7400–14100): 界提升证书、目标传播、隐含界、简约成本潜伏界、冲突学习; 根节点伪成本播种 (:5338–5400);
7. 规模自适应的割参数调优 (:14006–14100);
8. 根节点割平面循环 (:14292–14800), 节 6;
9. 根节点原始启发式 (:16980–22170), 节 7.3;
10. 根节点探测 (:22174–22320);
11. 搜索树搭建: 持久 LP 状态 (:15295–15315), 不可变的已缩放 `tree_base_sf` (:23689–23695), `NodeQueue` + 根节点入队 (:23447–23455);
12. 早期间隙检查 (:33926–33978), 节 10;
13. 串行循环 (:34091–35645) 或并行监控器 (:35766–36020);
14. 还原 (PaPILO :36431, 原生:36488) 并返回。

4 预求解

4.1 接线: PaPILO 优先, 原生预求解作为后备

原生预求解并非叠加在 PaPILO 之后; 它是一个后备替代方案 (`bac.cpp:2967--3049`): PaPILO 先运行 (`use_papilo_presolve{true}`, `options.hpp:162`); 原生 `MILPPresolve` 顶层遍历仅当 `!papilo_model_reduced` 时才运行 (`bac.cpp:3025`), 并额外受环境标志 `MIPSOLVERS_NATIVE_TOPLEVEL_PREOLVE_OFF` 门控。PaPILO 封装 (`src/engine/strategy/papilo_presolve.cpp`) 与 `MILPPresolve` 不共享任何代码。后果: 在 PaPILO 哪怕只缩减了一点内容的实例上, 下述原生约减都永远不会执行——它们实际上是冷备代码, 却仍然被发布、维护并纳入基准测试。

4.2 原生预求解：表示与约减

内部存储为统一的区间行形式 $\ell^{\text{row}} \leq a^{\top}x \leq u^{\text{row}}$ ，行向与列向关联表分别为 `std::vector<std::vector<Entry>> rows_` 和 `std::vector<std::vector<CEntry>> cols_` (`include/mipsolvers/engine/solver/native/milp/bc/milp_presolve.hpp:124,131`)，由 Eigen 矩阵的行主序副本构建 (`milp_presolve.cpp:81,98`)。并保留缓存的逐行活动度 $\{\underline{\text{act}}, \overline{\text{act}}, n^{-\infty}, n^{+\infty}\}$ 。容差: `zero_tol` = 10^{-10} , `bound_tol` = 10^{-9} , `max_rounds` = 20 (`milp_presolve.hpp:53,70--71`)。

约减清单 (均位于 `milp_presolve.cpp`; 数学已核验):

1. **固定变量移除** (:238–248): $|u_j - \ell_j| < 10^{-9}$; 对 `cols_[j]` 中的每一行做右端项平移 $b_r \leftarrow a_{rj}v$; 压入撤销记录。此处不更新行活动度。
2. **空行/空列** (:254–297): 空行仅当 $0 \in [\ell_r, u_r]$ 时才删除; 不可行的空行被静默保留 (“let LP detect it”, :262–265)。空列被固定到成本最优的界。
3. **单元素行** (:303–370): $ax_j \in [\ell_r, u_r]$ 蕴含界 ℓ_r/a 、 u_r/a ($a < 0$ 时翻转方向), 整数取整为 $\lceil x - 10^{-9} \rceil / \lfloor x + 10^{-9} \rfloor$; 随后删除该行 (隐含自由变量论证成立)。
4. **单元素列 (仅等式)** (:376–473): 代入 $x_j = (b_r - \sum_{k \neq j} a_{rk}x_k)/a_{rj}$; 被消去变量的界转化为残差行上的一个区间带; 成本转移 $c_k \leftarrow (c_j/a_j)a_k$ 。不等式单元素列明确 不处理 (:464–470)。
5. **双元等式代入** (:479–633): 从 $a_1x_1 + a_2x_2 = b$ 消去一个变量, 优先选择连续变量, 否则选择行出现次数更少者 (:517–527; :501–502 处的注释说的是“more”——注释与代码不符)。整数–整数消去仅当系数比值在 `bound_tol` 内为整数时进行 (:531–536)。
6. **基于活动度的界收紧** (:639–741): 对每个非零元 $u'_j = (u_r - (\underline{\text{act}}_r - \text{contrib}_j))/a_{rj}$ 等, 要求无穷贡献者数为零, 另有一个特殊的单无穷贡献者情形, 会对每个候选非零元重新扫描整行 (:699–736) ——长度为 k 的行复杂度为 $O(k^2)$ 。
7. **强制行/冗余行** (:747–807): 若 $\overline{\text{act}}_r \leq u_r$ (且下侧安全) 则冗余; 若 $\underline{\text{act}}_r \geq u_r - \text{tol}$ 则在最小活动度一侧强制, 将每个变量固定到其最小贡献界。
8. **“被支配”列** (:813–873): 仅按成本符号固定 ($c_j > 0 \Rightarrow$ 若不危及任何行侧, 则固定于 ℓ_j)。这并非成对列支配检验。
9. **平行行** (:879–968): 按列模式做哈希; 在同一桶内成对比较, 且每对都要对双方的非零元列表做一次复制 + 排序 (:913–923); 以相对容差做比例系数检验; 合并时按比值缩放界, 比值为负时翻转方向。仅在 `round < 3` 时运行 (:1441)。
10. **系数强化** (:974–1013): 仅针对 $\leq$ 行中 $a_j > 0$ 的二元变量做大 M 约减: $a'_j = u_r - \underline{\text{act}}_r^{\setminus j}$, 当 $0 < a'_j < a_j - \text{tol}$ 时应用, 同时 $u_r \leftarrow (a_j - a'_j)u_j$ 。没有负系数、一般整数或下侧变体。
11. **二元探测** (:1019–1165): 候选 = 未固定的二元变量; `max_probes` = $\min(500, \#cand)$, 若 $m > 2000$ 削减至 100, 若 $m > 5000$ 削减至 50 (:1033–1037); 硬性墙钟上限为 **1.0s** (:1040–1047)。每次探测都对 整个界向量做快照 (:1053–1054), 在包含 j 的行上传播 `probing_depth` = 3 轮 (:1071–1108)。两侧均不可行 $\Rightarrow$ `return count`, 且不设置任何不可行标志 (:1132–1135); 一侧不可行 $\Rightarrow$ 固定; 否则在全部 n 列上对两个探测世界求交: $\ell = \min(\ell^0, \ell^1)$, $u = \max(u^0, u^1)$ (:1147–1161)。探测中发现的蕴含关系被丢弃。

主循环 (`run()`, :1413–1485): 每轮按序执行——固定变量 $\rightarrow$ 空行/空列 $\rightarrow$ 单元素行 $\rightarrow$

单元素列 → 界收紧 → 固定变量 → 强制行 → 固定变量 → 空行/空列 → 二元等式 → 被支配列 → 平行行 (轮次 < 3) → 系数强化 → 探测 (轮次 ≥ 1) → 固定变量 → 空行/空列; 以变更计数器之和判定不动点, 硬性上限 20 轮, 除探测的 1s 外无时间限制。PresolveStats 没有不可行性字段 (milp_presolve.hpp:33--49)。

发现 1 (预求解的活动度维护是非增量式的)。每轮至少三次完整的 $O(\text{nnz})$ 活动度扫描 (≤ 20 轮): `tighten_bounds` 重算全部活动度 (milp_presolve.cpp:641), `detect_forcing_rows` 重算全部 (:749) 外加一次逐行刷新 (:759), `strengthen_coefficients` 重算全部 (:977); `fix_variable` 从不触碰活动度 (:170–196)。在 HiGHS/PaPILO 只做 $O(\Delta)$ 工作表更新的地方, 这里最多要做 ~ 60 次全矩阵扫描——而代码库已经拥有一个增量域 (`BCDomain`), 预求解却没有使用。头文件宣称“tracks row/column nonzero counts for $O(1)$ singleton detection” (milp_presolve.hpp:16--17); 这样的计数器并不存在——单元素检测是重新扫描 (milp_presolve.cpp:309--318, 382--391)。

发现 2 (探测学不到任何结构性信息, 且每次探测复制 $O(n)$)。每探测一个二元变量: 两份大小为 n 的快照、恢复复制、两份大小为 n 的输出副本、再次恢复, 然后一次 $O(n)$ 的求交扫描 (milp_presolve.cpp:1053--1161) ——无论受影响集合大小如何, 每次探测约复制 $\sim 5n$ 个 double; 蕴含关系被丢弃; 再加上 1s 的墙钟上限和 $m > 5000$ 时的 50 次探测上限, 使得探测在 MIPLIB 规模的实例上形同虚设。

发现 3 (预求解丢失不可行性并退化区间行)。双侧均不可行的探测返回时不带任何状态位 (milp_presolve.cpp:1132--1135) ; 不可行空行被跳过 (:262–265) ; 可证明不可行的实例会继续进入 LP/BC。重建时, 每个双侧行变成两个 $\leq$ 行 (milp_presolve.cpp:1197--1200, 1228--1246), 行数翻倍, 且行映射只记录上侧 (:1234, 1244) ——区间结构在下游丢失。

4.3 经核实缺失的标准约减

经 grep 核实确实缺失的内容 (均在检索之后才断言): 预求解中无对偶/简约成本固定 (它仅存在于节点级, bc_utils.cpp:1405, options.hpp:270); 无稀疏化; 无平行/重复列检测; 无成对被支配列检验; 不会将探测派生的蕴含插入蕴含图 (对比 HiGHS/PaPILO); 无单元素不等式列消去; 无针对负系数或一般整数的系数强化。PaPILO 覆盖了其中若干项, 但参见 §4.1: PaPILO 运行时原生遍历会被跳过, 反之亦然。

5 LP 松弛层

5.1 三层嵌套表示, 逐节点拷贝流转

- LPModel (problem_types.hpp:127--146): CSC A 、 A^{eq} 、稠密 c , 外加一份镜像的 HiGHS CSR 拷贝 (highs_row_start/index/value, :142–145)。
- StandardFormLP (dual_simplex.hpp:273--313): 矩阵存储 两次——CSC A 以及行主序 A_{row} (:277–278) ——外加松弛/人工列、lb_shift、var_ub、Ruiz 缩放因子, 以及用于 $O(1)$ 持久模型校验的 structure_id。

- HiGHS CSC 数组，每次求解由 `pass_standard_form_to_highs` (`dual_simplex.cpp:627--680`) 重建：全新的 `col_cost/lower/upper`、`row_lower/upper`、`start/index/value` 向量，用 `InnerIterator` 遍历所有列，然后 `highs.passModel(...)`——其内部还会再拷贝一次。

搜索树保存一份不可变的已缩放 `tree_base_sf` (`bac.cpp:23689--23695`) 和一个共享的可变 `node_sf` 工作区。

5.2 界变更：SF 层是增量的，在求解器边界被丢弃

`update_standard_form_bounds` (`dual_simplex_api.cpp:855--971`) 在每个子节点 (`bac.cpp:30719`) 和每次探测时被调用。它扫描全部 n 列以检测变更（内联缓冲 64；阈值 `kIncrementalThreshold= 50, :865`），当变更数 ≤ 50 时对 b 应用 $O(\text{nnz}_j)$ 稀疏列更新 (:906–920)，否则回退为一次完整的 $A \ell^{\text{shift}}$ 矩阵乘 (:923–947)，重写全部 n 个 `var_ub` 条目 (:952–959)，并更新目标常数项。接受已知变更的快速路径 `update_standard_form_bounds_incremental` (`dual_simplex_api.cpp:976`) 确实存在——却没有任何调用点（经 `grep` 验证）。于是每个子节点都要付出一次 $O(n)$ 检测扫描加一次 $O(n)$ 的 `var_ub` 重写，尽管分支只改动了一对界，而且 B&C 层明确知道改的是哪一对。

5.3 逐节点求解：全新 HiGHS，仅状态的热启动

发现 4（持久 LP 机制存在、已启用，随后却被中和）。默认的节点求解 (`dual_simplex.cpp:1337--1410`):

```
if (opt.allow_persistent_lp_state && basis_hint &&
    basis_hint->cached_sparse_basis &&
    basis_hint->cached_sparse_basis->resolve_same_structure(sf, basis_hint, opt, out))
    return true;                                     // dual_simplex.cpp:1345-1350
...
auto highs = std::make_shared<Highs>();             // :1354  <- fresh solver per node
highs->setOptionValue("presolve","off"); "solver","simplex";
               "simplex_strategy",1 /*dual*/; "threads",1; // :1357-1361
pass_standard_form_to_highs(*highs, sf);           // :1394  full  $O(n+m+\text{nnz})$  serialization
highs->setBasis(hbasis);                             // :1397-1400  statuses only
highs->run();                                          // :1402  cold INVERT first
out.form = sf;                                       // :1406  full SF deep copy per solve
```

一条真正的增量路径 `VendoredHighsBasis::resolve_same_structure` (`dual_simplex.cpp:1095--1210`) 会校验 `structure_id`、对所有列/行界做差分、仅对变更的条目调用 `changeColsBounds` 与 `changeRowsBounds`、在基未变时跳过 `setBasis`，并连同其因子分解一起复用存活的 `Highs` 对象——这正是 *SCIP/HiGHS-MIP* 的设计。串行调度器明确选择了启用 (`allow_persistent_lp_state = true`, `bac.cpp:23814`)。但是，每次节点求解之后，所存储的基提示由 `persist_bc_node_basis_from_simplex` (`bc_legacy_helpers.cpp:191--208`) 生成，它会执行

```
// Tree/node SF objects are per-node workspaces. Do not persist live BasisOps
// pointers tied to those temporary matrices across queue siblings.
```

```
basis->cached_sparse_basis.reset(); // bc_legacy_helpers.cpp:203-205
```

——在全部十一处树更新点 (`bac.cpp:28526, 28656, 31118, 31257, 31760, 32550, 32665, 33150, 33669, 33769, 34322`——经 `grep` 验证), 而 `Node::branch_child()` 拷贝的正是这份被剥空的提示 (`bc_types.hpp:366`)。因此, 每个常规树节点都无法通过:1345 处的门槛检查, 只能付出全新 `Highs + passModel` + 完全冷因子分解的代价。持久路径只可能对强分支的探测热启动子节点触发 (`bac.cpp:30697--30713`)。并行调度器从不启用 (`bc_parallel.cpp:280--281`; 默认 `allow_persistent_lp_state{false}`, `bc_solver_dispatch.hpp:68`)。(U) 这一 `reset` 究竟是正确性变通 (注释声称为了兄弟节点安全) 还是未完成的管线, 从代码本身无法判定; 启用开关的存在与完整实现的 `resolve_same_structure` 暗示后者, 但按现状, 全新重建就是事实上的节点 LP 算法。

基热启动在发生时仅传递状态 (`native_sf_basis_to_highs, dual_simplex.cpp:558--599`) : HiGHS 在第一次对偶迭代前必须重新因子分解 (冷 INVERT) ——因子分解、eta 文件、DSE 权重均不跨节点复用。存在一项部分缓解: `SimplexBasis::try_share_indices_from` (`dual_simplex.hpp:258--270`) 将相同的基索引向量去重存入共享的不可变存储, 这是一个真正不错的内存优化。

强分支探测对每个候选变量的每个方向都深拷贝整个标准形 (`bac.cpp:35017--35019: probe_sf = tree_base_sf`;——两种矩阵朝向和全部向量一并拷贝), 然后在全新的 HiGHS 上求解。

发现 5 (每次求解物化 $O(m^3)$ 稠密基逆, 且在该路径上是死重). 当 $m \leq 4096$ 时, `vendored` 封装层物化 `out.basis_inverse = Eigen::MatrixXd::Zero(m,m)` 外加 m 次 `getBasisInverseRow` 调用 (总计 $O(m^3)$, $m = 4096$ 时达 128 MB) (`dual_simplex.cpp:1602--1618`); 而证书路径在 `cached_sparse_basis` 存在时总是优先走稀疏 `BTRAN` (`dual_simplex.cpp:486--489`), 持久导入路径则显式跳过它 (`dual_simplex.cpp:1036`)。该稠密矩阵在默认路径上纯属死重——它还会撑大每一条 `CutRequest` 消息 (§9)。

5.4 向 LP 添加割与重新求解

根节点割插入按批次从三元组重建整个矩阵: `add_rows_to_lp / add_sparse_rows_to_lp` 把全部现存非零元流入一份新的三元组列表并调用 `setFromTriplets` ($O(\text{nnz} \log \text{nnz})$ 的分配 + 排序, `bc_utils.cpp:762--835, 837--902`; 三元组预留使用硬编码的每行 20 非零元猜测值, :798)。唯一真正增量的路径是根节点的实时追加事务 (`DirectHighsLpBasisOps, bc_relaxation.cpp:1121--1274`): 只打包新行、对列界做差分、`highs->addRows(...)`、热启动 `run()`、提交/回滚——但对任何含等式行的模型都会被拒绝 (“equalities not supported by live append”, :1138–1139), 且仅在根节点使用。节点上的割后重解 (默认关闭, 见 §6.9) 使用一次冷的完整 IPM 求解 (`bac.cpp:33309`), 而非热的对偶单纯形。

5.5 容差、缩放与迭代上限

核心: `int_tol= 10-5, gap_tol= 10-4, lp_tol= 10-6, max_lp_iter= 500, max_nodes= 50 000, time_limit_sec= 120` (`options.hpp:89--94`)。调度器的节点 LP: `feas_tol`

$= \text{opt_tol} = \max(1e-10, \text{lp_tol} * 0.1) = 10^{-7}$, $\text{max_iter} = \max(10 * \text{max_lp_iter}, 2000) = 5000$ (`bc_solver_dispatch.hpp:79--81, 211--212`); 随后 `vendored` 层再乘以十 ($\rightarrow 50000$, `dual_simplex.cpp:1363--1364`) , 而直接根路径使用 $\text{max_lp_iter} * 50$ ($\rightarrow 25000$, `bc_relaxation.cpp:1306--1307`)——跨层存在三套互不一致的迭代上限约定。节点可行性接受容差 $\text{kNodeFeasibilityTol} = 10^{-6}$ (`bc_utils.hpp:73`); 现任解剪枝容差 $\text{kIncumbentPruneTol} = 10^{-12}$ (`bc_utils.hpp:70`); 节点剪枝使用 $\max(10^{-9}, \text{lp_tol})$ (`bac.cpp:34230`)。Ruiz 缩放只对 `tree_base_sf` 施加一次 (`bac.cpp:23693`); HiGHS 内部缩放为 `simplex_scale_strategy=2` (`dual_simplex.cpp:1361`)。每个节点 LP 还要在 HiGHS 自身的 KKT 检查之上跑两份重复的 $O(\text{nnz})$ 残差审计 (`audit_vendored_highs_sf_result`, `dual_simplex.cpp:682--705`, 以及 `sf_solution_residual_acceptable`, `dual_simplex.cpp:2431--2499`)。

5.6 失败处理与回退层级

结果分类 (`classify_lp_result`, `bc_fallback.hpp:70--105`) 部分依赖于对人类可读状态字符串的子串匹配 (“Objective cutoff”、“unbounded”、“max_iter”——:78, 88–97): 内核消息一变就会脆断。回退层级 (`bc_fallback.hpp:248--320`; `enable_lp_fallback{true}`, `options.hpp:339`): **L5** 仅在 Farkas 证书确认的不可行/无界时剪枝; **L1** 至多 3 次重试, 每次深拷贝整个 SF 并将整个右端项抖动 10^{-7} (`perturbed_sf = sf`; , :274, 324–332)——而且扰动后 LP 的最优解被直接接受为节点结果, 不与未扰动模型做任何核对; **L2** 冷启动重解, 迭代数 $2\times$; **L3** 2 次重试, 容差放宽至 $10\times/20\times$ 、迭代数 $3\times$; **L4** 推迟该节点 (重新入队, 绝不丢弃)。最坏情形: 一个失败节点额外付出六次完整 LP 求解, 每次都在全新的 HiGHS 上。搜索树在剪枝前还会用一次含 Farkas 复核的完整 IPM 求解对失败做交叉验证 (`bac.cpp:30938--30969`)。

6 割生成

6.1 派发与 All 套件

唯一的派发点是 `add_cuts()` (`bc_cuts.cpp:8124--8335`); 默认为 `cuts{CutType::All}` (`options.hpp:108`)。All 套件按如下顺序触发 (`bc_cuts.cpp:8251--8334`): (1) 变换单纯形表 cMIR (“Gomory”, 占全部剩余预算, 需要单纯形基); (2) “basis MIR”, 预算按规模为 $4/2/0$; (3) 行 MIR, $5/3/2$; (4) 背包覆盖, $5/3/2$; (5) 来自持久团表的团割, $5/3/2$ (仅当 $n \leq 10000$ 时才有即时回退); (6) 隐含界, $3/2/1$; (7) 零半割, 5, 仅当 $n \leq 10000$ 。预算通过 `spend_tracked` (`bc_cuts.cpp:8173--8200`) 共享, 后者喂给自适应割平面族门控 (§6.7)。规模阈值: $\text{large} = n > 10^4$, $\text{xlarge} = n > 2 \cdot 10^4$ (`bc_cuts.cpp:8257--8258`)。流覆盖被有意排除在 All 之外:

```
// Flow-cover separation is intentionally not part of the generic All bundle.
// The legacy implementation only infers capacity structure from signed row
// coefficients; on presolved/ranged rows this can produce rows that cut off
// the true integer frontier.          bc_cuts.cpp:8328-8332 (verbatim comment)
```

即作者自己标出了一处有效性缺口, 并让该割平面族仅作诊断用途。另外两个割实现是死的: 经典的互补单纯形表 MIR 被编译期剔除 (`bc_cuts.cpp:303: #if 0 // MIR cuts disabled:`

correct but hurt tree search performance.),而经典的 GMI 路径(build_bounded_form_gmi_cut, :120–301; add_basis_gomory_cuts, :6193–6350) 没有调用者——实际在用的 “Gomory” 是下文那种 HiGHS 风格的变换 cMIR。

6.2 变换单纯形表 cMIR (“Gomory” 路径)

add_transformed_tableau_cuts (bc_cuts.cpp:6352--6881)。**来源**：对每个分数取值的整数基变量，用 BTRAN 恢复单纯形表行， $y^\top = e_i^\top B^{-1}$ (:6536–6555; 经 vendored-HiGHS BasisOps 走稀疏路径，稠密回退为 simplex.basis_inverse.row(i)，位于:6551)，模型行按权重 $w_r = y_r \cdot \text{btran_scale} \cdot \text{row_scale}$ 在稠密 $n+m$ 向量上以扩展精度累加器聚合 (XTabLpAggregator, :4232–4335)；行端通过带界的显式松弛列进入，故聚合不等式的右端恒为 0 (:4274–4292)。行选择按 $\text{score} = f(1-f)/\|w\|^2$ (:6606) 打分，并做相对权重过滤；若 $\max_r w_r / \min_r w_r > 10^4$ 或条目数 ≤ 1 则拒绝该行；候选数上限为 $\min(5000, 200 + 0.1 \min(m, n_I))$ (:6496–6507)；当得分跌破最佳值的 $0.0025\times$ 时提前停止（接受 50 条割后为 0.01, :6703–6709）。

割推导 (xtab_generate_cut_from_base_row, :3992–4222)：(i) 界变换——每一项都相对其最近的界来表达，包括变量界 $x \geq c + \alpha y$ / $x \leq c + \alpha y$ ，其二元触发变量选自蕴涵图/VLB 表 (:2704–3184)；(ii) 预处理——二的幂预缩放、连续项消去、行长上限 $100 + 0.15n$ (:3186–3307)；(iii) 先试一条提升覆盖割，记录其效能 e_c ，再以抬高的门槛 $\text{min_efficacy} + e_c$ 运行 cMIR 并保留较优者 (:3535–3625)；(iv) cMIR (:3309–3533)：对候选缩放 δ （整数项的互异 $|a_j|$ 外加细化值），计算

$$\bar{a}_j(\delta) = \left\lfloor \frac{a_j}{\delta} \right\rfloor + \max\left(0, \frac{f_j - f_0}{1 - f_0}\right), \quad f_0 = \left\{ \frac{\text{rhs}}{\delta} \right\}, \quad (3)$$

效能 = 违反量 / $\sqrt{\text{norm}^2}$ ，接受最优的 δ ，然后尝试 $2\delta, 4\delta, 8\delta$ 以及逐项的互补翻转 (:3440–3492)；守卫条件 $f_0 \in [0.005, 0.995]$ 、 $(1 - f_0)^{-1} \leq 10^6$ (:3318–3320)；(v) 经由精确的标准形系数做逆变换 (:3627–3762)，再后处理（小系数消入右端、经 HighsIntegers::integralScale 做整数缩放、右端向下取整、对类整数列做系数收紧 $|a_j| \leq \text{maxact} - \text{rhs}$, :3778–3990)。最终准入要求违反量 $> 10 \cdot \text{feastol}$ (:4208)。每条聚合行的两种符号都会尝试 (:6743)。

6.3 路径聚合、路径混合与模 k 割

add_transformed_path_cuts (bc_cuts.cpp:6883--7738) 是 HiGHS HighsPathSeparator 的对应物：行按活跃度被分类为 Eq/Leq/Geq/Unusable；恰有一个远离界的连续列的等式行成为替换弧；从每个可用起始行出发，分离器扩展出一条长度 $\leq \text{kMaxPathLen} = 6$ 行的路径 (:7426)，通过 $w = -\text{val}/\text{arc.coeff} \in [\text{feastol}, 1/\text{feastol}]$ (:7427–7431) 消去离界最远的连续列；每个前缀都以两种符号走一遍 cMIR 管线。路径混合 (xtab_generate_path_mixing_cut, :4343–4712) 收集右端严格递减的取负前缀聚合，要求路径行间逐列的界类型一致，取 $\delta = 2^{\lceil \log_2(\max_j |a_j| + 1) \rceil}$ ，并增量地构建混合割，在第一个违反量 $> 10 \cdot \text{feastol}$ 的前缀处产出。模 k 割 (add_transformed_modk_cuts_impl, :4767–5244)：活跃行被缩放为整数（缩放为 0 或 $> 10^6$ 则拒绝），组装成 $\mathbb{Z}$ 上的 CSC 方程组，并依次在 GF(2)、GF(3)、GF(5)、GF(7) 上经 HighsGFkSolve 求解 (:4745–4765)，在第一个产出行被接受的模数处停止；每个解权重向量生成两条聚合行 (w/k 与 $(w(k-1) \bmod k)/k$)，送入 cMIR 管线。

6.4 模型行割平面族

- **背包覆盖** (`add_cover_cuts`, :5307–5456): 取 $b_r > 0$ 、系数全正、支撑全为二元变量的行; 按系数递减贪心构造覆盖, 直至 $\sum w_j > b_r + 10^{-9}$; 割为 $\sum_{j \in C} x_j \leq |C| - 1$ (:5390–5392); 对非覆盖变量做序贯上升提升 (仅限候选数 ≤ 40 的行), 使用 **离散化背包 DP**, **N_BINS**= 512 个桶 (:5410–5439), $\alpha_k = \text{rhs} - z^*$; 准入违反量 $> 10^{-7}$ 。该族没有密度/效能/平行度过滤。见发现 7。
- **cMIR 管线内部的提升覆盖** (:1478–1941): 经注水式 **abar** 循环构造精确的超可加提升函数 (:1566–1585), 带半整数检测; 经部分覆盖和上的分段线性 $\phi(a)$ 做混合二元提升; 经双侧超可加的 ϕ_l/γ_l 做混合整数提升 (:1842–1915)。这是精确的机制——但仅在单纯形表管线内部使用, 不用于模型行覆盖。
- **流覆盖** (`add_flow_cover_cuts`, :5662–5795): 对含负二元系数的二元 + 连续混合行, $\lambda = b_r + \sum_{N^-} |a_j|$; 按 $|a_j|(1 - x_j^*)$ 贪心构造覆盖 $C \subseteq N^-$, 直至 $\sum_C |a_j| > \lambda + 10^{-9}$; 不等式在 N^+ 上取 $\min(a_j, \lambda)$, 在覆盖上取 $\max(0, |a_j| - \text{excess})$ (:5740–5768)。这是一条简化的单覆盖不等式: 无序列无关提升, 无 Gu–Nemhauser–Savelsbergh 式的 L^- 处理, 并因上文引述的有效性理由被排除在 **All** 之外。
- **隐含界** (`add_implied_bound_cuts`, :5956–6056): 见发现 6。
- **行 MIR / “basis MIR”** (`add_row_mir_cuts` :6060–6171, `add_basis_mir_cuts` :5458–5655): 两者都作用于模型行而非单纯形表; “basis” 变体把它的 `simplex/sbasis` 参数标记为 `[[maybe_unused]]` (:5460–5463) ——名称属于误称, 这两个几乎相同的实现在同一轮 **All** 中以不同过滤器运行。算法: 对 $x_j > (\ell_j + u_j)/2$ 的整数变量取互补, 要求 $f_0 = \{b'\} \in (10^{-8}, 1 - 10^{-8})$, 整数项的 MIR 系数在 $f_j \leq f_0$ 时为 $\lfloor a_j \rfloor$, 否则为 $\lfloor a_j \rfloor + (f_j - f_0)/(1 - f_0)$; 负连续 a_j 取 $a_j/(1 - f_0)$ (:5538–5558, :6109–6119)。只有 “basis” 变体带效能下限 ($5 \cdot 10^{-4}$)、密度上限 (按 n 为 0.60/0.30/0.10) 和一个 $O(k^2)$ 的 $|\cos| > 0.90$ 平行度过滤器 (:5636–5643); 行变体完全没有过滤 (:6156–6170)。
- **零半割** (`add_zero_half_cuts`, :7928–8122): Caprara–Fischetti $\{0, \frac{1}{2}\}$ -割, 限于奇支撑相同的行对 (mod-2 签名 + FNV-1a 指纹分组, 每组 ≤ 30 行 $\Rightarrow \leq 435$ 对, :7988–8032), 松弛窗口 $-10^{-6} \leq s \leq 0.99$, 违反量估计 $0.5 - 0.5(s_1 + s_2) \geq 10^{-4}$; 割为 $\frac{1}{2}(a_{r_1} + a_{r_2})^\top x \leq \lfloor (b_1 + b_2)/2 \rfloor$, 整数变量系数经取整 (:8073–8091); 对已产出割做平行度过滤 $|d| > 0.90 \|c\| \|c_i\|$ (:8098–8110)。通用的 $\text{GF}(k)$ 机制就在同一文件中, 但此处未复用。
- **团割** (§6.5)。

发现 6 (隐含界“割”与其源行在代数上完全相同). 对一个二非零行 $cx + dy \leq b$ ($c > 0$, y 为二元变量), 分离器计算 $u_0 = b/c$ 、 $u_1 = (b - d)/c$ (`bc_cuts.cpp`:6007--6010, **V**) 并产出

$$x - (u_1 - u_0)y \leq u_0 \quad (\text{bc_cuts.cpp:6024--6030}). \quad (4)$$

代入定义: $x - (\frac{b-d}{c} - \frac{b}{c})y \leq \frac{b}{c} \iff x + \frac{d}{c}y \leq \frac{b}{c} \iff cx + dy \leq b$ ——恰好就是源行除以 c 。在任何 LP 可行的 x^* 处, 其违反量至多为 LP 可行性容差, 故 $> 10^{-7}$ 的准入测试 (:6032–6033) 只能靠数值噪声通过——而一旦通过, LP 就多了一条经缩放的、与现有模型行重复的行。该割平面族在 **All** 套件的每一轮根节点割中都运行 (`bc_cuts.cpp`:8314--8316)。

发现 7 (覆盖 DP 提升悄无声息地近似了提升系数). 模型行覆盖提升按 $\lfloor w \cdot 512 / \text{cap} \rfloor$ (`bc_cuts.cpp:5415--5430`) 离散化权重: 离散化权重为 0 的物品以零容量代价 进入背包 (:5423), 这会高估 z^* , 从而使 $\alpha_k = \text{rhs} - z^*$ 低于真实提升系数——一个弱于有效值的系数 (离散化方向向下意味着割仍然有效, 但可能任意弱)。尽管同一文件中就有精确的超可加提升机制 (:1552–1641), 这里却没有精确 DP。候选数 > 40 的行完全跳过提升 (:5397)。

6.5 团割与持久团表

即时分离 (`add_clique_cuts`, :7747–7915): $m > 15000$ 时跳过; 冲突边来自二元变量数 ≤ 200 的纯二元 $\leq$ 行, $a_i + a_j > b + 10^{-9}$ ——对每行做一次 $O(\sum_r k_r^2)$ 的两两扫描 (:7785–7795); 邻接结构为 `std::vector<std::vector<int>>`; 取 ≤ 200 个满足 $0.1 < x_j^* < 0.9$ 的种子; 按 x 最大的候选做贪心极大团扩展; 若违反量 $> 10^{-7}$ 则产出割 $\sum_{j \in C} x_j \leq 1$ 。持久 `CliqueTable` (`bc_clique_table.cpp`) 在根节点构建一次 (`bac.cpp:6823`, `max_row_nnz=256`): 扫描 A 与 A^{eq} 行的两侧, 界平移后转换为正的文字系数, 当两个最大系数超过剩余容量时发出一条文字边 (最坏情形每行 $\binom{256}{2} \approx 32640$ 次成对测试, :184–199)。存储为双 CSR: $2n$ 个节点上的一个变量图和一个文字图 (`bc_clique_table.hpp:213--218`)。根节点各轮查询 `find_violated_cliques` (`xlarge` 时 ≤ 128 , 否则 512, `bc_cuts.cpp:8266`), 在 CSR 上贪心扩展并以二分查找判定成员关系 (:897–997)。

发现 8 (团表增量更新每次调用都从零重建两张 CSR 图). `add_edges/add_literal_edges` 把完整 CSR 分解为 `std::vector<std::vector<int>>`, 追加后对每个顶点列表重新排序/去重, 再重建两张 CSR (`bc_clique_table.cpp:311--446`) ——每次调用 $O(V+E)$ 的分配, 而且这每被接纳一条割行 就运行一次 (`bac.cpp:10618, 15433`)。HiGHS 的团表以逐顶点归并插入的方式追加进分块 CSR, 每条边 $O(\text{degree})$ 。

6.6 CGLP 提升-投影

默认关闭 (`enable_cglp_cuts{false}`, `options.hpp:503`)。析取: 单个二元变量 $x_j \leq 0 \vee x_j \geq 1$ (`bc_cglp.hpp:17--20`)。主 LP (`build_cglp_lp`, `bc_cglp_model.cpp:89--212`) 把源 LP 规范化为 $Mx \leq d$, 共 $R = m_{\text{ineq}} + 2m_{\text{eq}} + \#\{\text{finite bounds}\}$ 行; 变量为 $(\alpha \in \mathbb{R}^n, \beta, \lambda^0 \in \mathbb{R}_+^R, \gamma_0, \lambda^1 \in \mathbb{R}_+^R, \gamma_1)$, $n_{\text{vars}} = n + 2R + 3$; 约束为 $\alpha + M^\top \lambda^0 + e_j \gamma_0 = 0$, $\alpha + M^\top \lambda^1 - e_j \gamma_1 = 0$, $\beta + d^\top \lambda^0 \leq 0$, $\beta + d^\top \lambda^1 - \gamma_1 \leq 0$, 以及乘子求和规范化的 $\sum \lambda^0 + \gamma_0 + \sum \lambda^1 + \gamma_1 = 1$ (硬编码; `cglp_normalization{0}` 存在但未使用, :171–179); 目标 $\min \alpha^\top x^* - \beta$ (:119–123)。每轮对至多 `cglp_max_candidates{5}` 个最分数的二元变量, 各由内部单纯形冷启动求解一次, `max_iter=40000` (`bc_cglp.cpp:263--274`)。插值回退每个候选最多花费 32 个辅助冷启动 LP (8 个变量 $\times$ 4 个侧向 LP, :49–109), 外加强化阶段的验证 LP (:158–220)。代码自己记录了这场膨胀: “*costs > 4 s/candidate and blows the time budget*” (`bc_cglp.cpp:292--294`)。仅靠默认关闭的开关和 2s 的轮次限制来缓解 (`cglp_time_limit_sec{2.0}`, `options.hpp:513`)。

6.7 割选择与过滤

- 效能处处为 $= \text{violation} / \|a\|_2$; 阈值: $\text{GMI gmi_min_efficacy} = 10^{-3}$ (`options.hpp:102`; 在类 SCUC 根节点上自动放宽至 10^{-4} , `bac.cpp:14079`), `basis-MIR` $5 \cdot 10^{-4}$, `cMIR` 内

部门槛 $10 \cdot \text{feastol}$; 模型行各族按未归一化违反量 $> 10^{-7}$ 准入 (覆盖:5443, 流覆盖:5772, 隐含界:6033, 行 MIR :6135, 团:7888) ——这一不一致让缩放糟糕的行占据主导; SCIP 则完全按效能过滤。

- **平行度**: $|\cos| = |a^T b| / (\|a\| \|b\|)$ (`abs_cosine_similarity`, :478–487), 在单纯形表/路径/零半/basis-MIR 的选择中, $\geq \text{gmi_max_parallelism} = 0.90$ (`options.hpp:104`) 即拒绝; 覆盖、流覆盖、隐含界、行 MIR 与团割各族**没有**跨割正交性过滤。
- **密度**: $\text{nnz} \leq \lceil 0.35n \rceil$ (`gmi\_max\_density`, `options.hpp:103`), 在 very-large/large 根节点上自动收紧至 0.10/0.15 (`bac.cpp:14028,14034`), 在类 SCUC 上放宽至 0.50 (:14080)。
- **GMI 排序得分** = $\text{efficacy} \cdot (1 + \text{gmi_activity_weight} \cdot \text{activity})$, `gmi\_activity\_weight` = 0.5 (`options.hpp:107`; `bc\_cuts.cpp:6773--6774`)。
- **自适应割平面族门控**: 对每族在 5 轮循环缓冲上的逐轮效能做滑动平均; 一旦轮数 ≥ 5 且滑动均值 $< 10^{-3}$, 该族即被永久禁用 (`CutFamilyStats`, `bc\_types.hpp:655--705`)。这是单向开关: 一个不走运的根节点阶段 (例如无基的 IPM) 会让 MIR/覆盖在整个求解中被禁用; 所度量的效能是重解前那一点的违反量——正是这些割被设计要切掉的那一点——而非界改进, 因此该门控奖励的是自我印证的割平面族。

6.8 割池机制

`PoolCut` (`bc\_types.hpp:317--327`): 一个拥有所有权的 `Eigen::SparseVector<double>` (私有堆缓冲)、`rhs`、`age`、`best\_efficacy` (声明时无初始化器, :321——潜在的不确定值缺陷; 目前所有聚合初始化点都给它赋了值, 故暂时蛰伏)、`norm`、`hash` 及标志位。`CutPool` (`bc\_pools.hpp:569--1002`): 容量 `cut\_pool\_max\_size{2000}`, `cut\_pool\_max\_age{50}` (`options.hpp:206--207`)。

发现 9 (割池: 哈希计算了却从未使用; 老化机制是死的; FIFO 驱逐). `add()` 计算 `sparse\_cut\_hash` (`bc\_pools.hpp:622, V`), 随后却以全池线性扫描来去重: 对每个候选测试支撑集相等外加一次稀疏点积 (:624–654) ——每次插入 $O(P \cdot \text{nnz})$, P 可达 2000, 而存储的哈希早已为 $O(1)$ 桶索引付过代价。`age\_and\_purge` (:976–994) 没有任何调用点 (经 `grep` 在 `src/` 上验证); 每条割出生即 `initial\_cut\_age() = max(0, 50-5) = 45` (:595) ——出生时已过期 90% ——而且由于年龄从不变化, 池满时总是经 `max\_element` (:656–665) 驱逐最早插入的割, 即盲目的 FIFO, 毫无基于效能的保留。`find\_violated` 是每节点一次无索引全扫描 (:955–973)。SCIP/HiGHS 使用哈希索引的池, 并按年龄/效能驱逐。

该池还兼任传播引擎 (`propagate\_with\_reasons`, `bc\_pools.hpp:687--951`); 其热循环对每条活跃割每轮做两次完整的稠密 $O(n)$ 拷贝 (:806–807), 并为构造原因对每个被收紧的列重新遍历整行 (最坏情形每条割 $O(\text{nnz}^2)$, :832–848), 每轮重扫整个池 (:923–948; 由一个可选的 `pending\_cols` 过滤器缓解, :724–741)。

6.9 根节点与树内割策略

根节点: 至多 `root\_cut\_rounds{10}` 轮 (`options.hpp:99`), 按规模缩减 ($m > 35000$ 时为 0, $m > 15000$ 或 $m > 12000$ 时 ≤ 3 , $m > 2000$ 时 ≤ 6 , `bac.cpp:14017--14040`, `options.`

hpp:840--843); 每轮预算 cuts_per_round{20} (options.hpp:100) ——按规模分层的每轮上限 (cuts_per_round_medium/large/very_large= 60/40/30, options.hpp:844--846) 在默认值下形同虚设 ($\min(20, 30) = 20$); 只有轮数真正有约束力。停滞退出: 连续 root_cut_max_stalls{2} 轮改进量 /denom < root_cut_stall_tol{1e-4} (bac.cpp:14788--14795)。在 root_cut_reject_nonmoving_rows{true} (options.hpp:382) 下, 界提升 $\leq$ root_cut_min_bound_lift{1e-6} 的一轮会被 整体回滚 (truncate_ineq_rows) 并跳出循环 (bac.cpp:14398--14430, 14740--14756) ; 各行会被审计有效性/是否切掉现任解 (root_cut_audit_validate_rows{true})。每轮还要为界传播收紧检查制作两份 A 与 A^{eq} 的完整行主序拷贝 (bac.cpp:14398--14405)。每轮重解: 以手工扩展的基热启动单纯形 (割松弛插入在 ineq/eq 边界处, :14536–14634), 大根节点则用 IPM。

树内: 节点割生成要求 child.depth <= max_cut_depth, 而 max_cut_depth{0} (options.hpp:101; bac.cpp:33259) ——树内分离默认是关闭的 (根节点的子节点深度 ≥ 1)。节点上的割池再分离被硬性门控在 $A.rows \leq$ pool_cut_row_threshold{500} 且深度 $\leq$ pool_cut_depth_limit{3} (bac.cpp:33179, options.hpp:550--551)。既没有 SCIP/HiGHS 那种基于频率的分离 (每 k 个节点一次), 也没有效能触发的节点分离; 出于同一原因, 割工作线程默认也是死的 (use_cut_worker = ... && opt.max_cut_depth > 0, bac.cpp:18096)。

7 分支、节点选择与原始启发式

7.1 伪成本模型与分支规则

默认 BranchingStrategy::Pseudocost (options.hpp:113; 备选为 MostInfeasible、FirstFractional, enums.hpp:12--16) 。对每个变量与每个方向, PseudoCost (bc_types.hpp:392--521) 跟踪和、平方和、计数、推断计数、截断计数与冲突分数。观测值为 Bénichou 归一化的单位增益

$$\psi_j^\pm = \frac{\Delta_{obj}}{\max(10^{-9}, |\text{frac_dist}|)} \quad (\text{normalized_pseudocost_gain, bc_types.hpp:525--528}), \quad (5)$$

由强分支探测与已实现的子节点 LP 共同更新 (update_pseudocost_from_branch_evidence, bc_branching.cpp:296--318)。选择时的估计为 95% 下置信界

$$\text{LCB} = \max(10^{-6}, \bar{\psi} - 1.96 \sigma / \sqrt{\text{cnt}}) \quad (\text{bc_types.hpp:430--444}), \quad (6)$$

未观测时取 $\bar{\psi} \equiv 1.0$ (:409–411), 历史乘子为 $1 + 0.05 \ln(1 + \text{inference}) + 0.25 \cdot \text{cutoff} + 0.05 \ln(1 + \text{conflict})$ (:471–477) 。方向得分: $|\text{dist}| \cdot \max(\text{LCB} + \text{offset}, 10^{-6}) \cdot \text{history}$ (bc_branching.cpp:159--187); 变量得分为 乘积 $s_j = s_j^- \cdot s_j^+$ (compute_branch_product_score, :189–199); 候选按 $\ln(1 + \max(0, s_j))$ 排序, 并叠加一层软分支先验 (:483–484)。方差采用教科书中已知不稳定的 $E[x^2] - E[x]^2$ 公式 (bc_types.hpp:414--426, 在 0 处截断) ——尽管:391 处有“Welford 风格”的注释, 对大规模累加和仍存在数值风险。

可靠性分支/强分支 (实际的默认行为)。在 Pseudocost 下, 当候选预算非零时, 搜索树在 depth ≤ 6 处执行带强分支探测的可靠性分支 (bac.cpp:34906): 一个变量在每个方向都

累计 $\geq \text{probe_reliability}\{2\}$ 次观测之前被视为“不可靠”（一旦两个方向都 ≥ 1 ，则减半至 $\max(2, r/2)$ ，:34910–34913）；最多探测 $\min(\text{probe_max}, |\text{unreliable}|)$ 个候选（根节点处 ≤ 3 ，树内 $\leq 1-2$ ，:34926–34928； $\text{probe_reliability}\{2\}$ 、 $\text{probe_max_candidates}\{3\}$ ，`options.hpp`:537–538）。每次探测都是对 `tree_base_sf` 的深拷贝做两次完整的单纯形求解（`bac.cpp`:35017–35029），且没有探测专属的迭代上限——HiGHS/SCIP 则是在现役 LP 上进行强分支，并设置较小的迭代限制与对偶界截断。被选中子节点的探测 LP 会被保留并直接用作实际的子节点 LP（`bac.cpp`:30902–30909，`strong_branch_cache_exact_hits`）；其余探测结果仅用于填充伪成本。探测还可能附带产生现任解（:34958–34989）。

方向与估计。 `choose_up_branch_first`（`bc_branching.cpp`:201–227）：当存在现任解且恰好有一个方向已被证明截断时，最后才分支到截断一侧；无现任解时，下降到预测得分较小的一侧。节点估计（`compute_node_estimate`，:969–1005）：

$$\text{est} = \text{bound} + \sum_{j \text{ fractional}} \min(s_j^-, s_j^+) \quad (\text{Sum aggregation, default; Maximum takes the max}). \quad (7)$$

7.2 节点选择

默认 `NodeSelection::Hybrid`（`options.hpp`:114）：在获得首个现任解之前采用 DFS，之后转为估计引导的最佳优先（`enums.hpp`:19–23）。优先级键（`bc_utils.hpp`:76–84，`V`）：

$$\text{key}(\nu) = \frac{1}{2}(\text{bound}_\nu + \max(\text{bound}_\nu, \text{est}_\nu)), \quad (8)$$

若下界节点近期未被弹出，则每 `kHybridBestBoundPeriod`=10 个叶子强制弹出一次最佳界节点（`bc_utils.hpp`:314–317,1865）。一旦存在证明有效的现任解（“证明尾部模式”），弹出策略切换为纯最佳界（`bac.cpp`:34199–34201，23468–23477）。

发现 10（开放节点队列：一个哈希映射加三个红黑树索引再加一个 `vector`，全靠手工保持一致）。顺序版 `NodeQueue` 将每个 `Node` 按值存储在 `std::unordered_map<int64_t, Node>` 中，外加三个 `std::multiset<pair<double, int64_t>>` 索引（`priority_`、`bounds_`、`dfs_bounds_`）以及一个 `std::vector<int64_t>` `dfs_`（`bc_utils.hpp`:1850–1854，`V`）。每次弹出/删除都要为每个索引支付 $O(\log N)$ ，而 DFS 侧的删除执行 `dfs_.erase(std::remove(...))`——即每次队列变更都要做一次 $O(|\text{dfs}|)$ 线性扫描（`bc_utils.hpp`:372，`V`；线程安全队列中亦如此，`bc_threading.hpp`:772,884）。`std::multiset` 为每个索引中的每个条目分配一个树节点——在二叉堆本已足够之处做指针追逐（`bc_utils.cpp`:1306–1354 中遗留的 `std::priority_queue` 渐近性能更优，且仍然存在）。每次压入还要额外构建一个 $O(\#\text{branchable})$ 的 `std::string` 域签名用于去重（`bc_utils.hpp`:1945–1986；线程安全孪生版见 `bc_threading.hpp`:709–750）——每个可分支列 ≈ 18 字节，并在每次节点插入时被哈希进另外两个哈希映射。并行版 `ThreadSafeNodeQueue` 在单一全局互斥锁加条件变量之下复制了同样的四索引设计（`bc_threading.hpp`:646–1141），其 `count_clause_hits` 还会在持有队列锁的同时对每个排队节点的稠密界数组执行一次 $O(N \cdot L)$ 扫描（:1105–1134）。

7.3 原始启发式

根节点启发式（由 `!root_gap_closed()` 及规模/现任解跳过标志门控；顺序依据 `bac.cpp:17984--17991` 处的泵窗口轨迹）：

1. **可行性跳跃** (:16980–17085; `enable_feasibility_jump{true}`, ≤ 512 次翻转, 3 次修复尝试, `options.hpp:195--197`)：通过二元翻转降低行不可行性, 并辅以固定整数 LP 修复。
2. **快速取整** (:17100–17243)：具备 UC 意识的承诺取整加锁定计数偏置（已关闭），随后运行固定整数修复 LP；若时间预算已消耗 $> 25\%$ 则跳过。
3. **线搜索中心取整** (:17246+；默认关闭)：HiGHS 风格的 `tryRoundedPoint`, 以解析中心为锚点。
4. **固定整数可行性泵** (:17373–17458; `use_feasibility_pump{true}`, $\leq$ `feasibility_pump_iters{5}` 次迭代, `options.hpp:359`)：每次迭代求解一个固定整数 LP。
5. **目标可行性泵** (:17461–17970)：面向 $m > 10^4$ 的 Fischetti–Glover / Achterberg–Berthold 风格；在热启动的原始单纯形基上切换 L1 距离目标；基于 Zobrist 哈希、历史长度为 3 的循环检测； α 重启；阶段 3 的重启上限。
6. **阶段 3 子 MIP** (:17840–17970)：在固定受限模型上以 `_is_stage3_submip` 递归调用 `branch_and_cut_lp`, 预算 0.25–2.0s。
7. **LP 潜泳** (:19360–20280; `max_dive_lps{60}`, `options.hpp:360`)：以约化成本引导的贪婪固定, 采用热启动对偶单纯形；大规模/超大规模根节点获得零个潜泳 LP, 除非低分数性救援条件成立 (:19403–19419)。潜泳循环每一步都重建并重新排序完整的分数候选向量 (:19482–19500) ——最多 500 步时复杂度为 $O(n \cdot \text{steps})$ 。
8. **渐进取整** (:18190–19245; `use_progressive_rounding{true}`)：结合约束传播的批量固定、自适应批量大小与成组翻转打磨。
9. **根节点 LNS** (:20300–20460; `enable_lns{true}`)：将前 `lns_fix_ratio{0.80}` 比例的分数变量固定在现任解取值处, 求解受限子 MIP (`lns_node_limit{500}`、`lns_time_limit{5.0}`, $\leq$ `lns_max_iters{3}`) ——且受限模型是**每次迭代一份完整 LP 拷贝** (`restricted.linear_part = base_lp;`, :20381)。
10. **内嵌 HiGHS 根子 MIP / RENS** (:20809–20960) 以及残差 RENS 前沿子 MIP (:21251–21760)。

树内启发式：逐子节点的取整检查与带预算的取整 LP 修复（按深度分级的迭代上限 50/150/500）；每 `crossover_heuristic_freq{100}` 个节点执行一次**交叉**，组合解池中最好的两个解；每 `rins_frequency{200}` 个节点执行一次**廉价 RINS**——仅做 LP/现任解一致取整；`bac.cpp:35549` 处的注释为一个近终止门控保留了“完整 RINS 子 MIP”，而该门控在树阶段**并不存在** (:35549 与循环结束:35645 之间没有此类代码)。现任解局部分支打磨默认关闭，仅在 $\sim 3 \times \text{gap_tol}$ 范围内作为终止打磨运行 (:29285–29400)。

8 域传播、冲突学习与目标传播

8.1 BCDomain: 增量传播器

BCDomain (`bc_domain.hpp`) 是真正意义上的 HiGHS HighsDomain 对应物: 稠密的 `lb_`/`ub_`、带无穷计数器的逐行缓存活动度、由 $(j, \ell^{\text{old}}, u^{\text{old}}, \ell^{\text{new}}, u^{\text{new}})$ 构成的轨迹, 以及标记行工作列表。`change_bound` 执行 $O(\text{nnz}_j)$ 的增量活动度更新 (:154–186)——渐近复杂度正确。它用于修复 LP 流水线, 但不用于树节点域 (树节点域是 `Node` 内部的完整拷贝)。

发现 11 (BCDomain 在回溯时破坏了自身的增量性, 并可能静默截断不动点). `restore()` 回滚轨迹后重新标记所有行 (`bc_domain.hpp:139--143, V`), 因此下一次 `propagate()` 实际上会重扫几乎所有行; `mark_propagate` 的可收紧性测试被硬编码为 `true` (:335–342: `// *tightenable*/ true`), 因此几乎每个有限界行在每次触碰时都有资格参与。而 `propagate()` 在访问 `max(64, 8*m)` 行后停止 (:194, `V`), 在仍有行排队的情況下返回成功——传播被静默地截断, 调用方无法察觉。HiGHS 则循环直至队列排空。

8.2 带理由跟踪的逐节点传播

`propagate_node_domain` (`bc_utils.cpp:6302--6350`, 实现位于:5764–6298) 是逐节点不动点引擎 (“每次节点 LP 求解前都要经过的热路径”, `bc_utils.hpp:3229--3233`): 调用方构建的 `RowPropagationIndex` (良好设计)、冲突池重放、按变更变量触发的团/蕴含弧、`max_passes = 2*rounds + 4` (默认 `bound_propagation_rounds{3} ⇒ 10` 趟)。

发现 12 (每个节点都要支付的 $O(\text{nnz}_{\text{row}}^2)$ 理由 `id` 收集与无界 arena 增长). 对于被访问行的每个非零元 j , 传播器都会重扫整行以收集理由 `id` (`bc_utils.cpp:6134--6141` 为不等式路径; :6199–6208 与 :6235–6244 为等式路径, 后者还在逐非零元循环内部声明 `std::vector<int> ids;`, 每个非零元一次堆分配, 与该文件自己在 :6123–6125 所做的提升修复相矛盾), 随后 `ReasonArena::merge_ids` 对合并后的文字排序/去重, 并每次追加一个全新的堆分配子句 (:5654–5672), 完全没有复用/去重——每行访问、每个非零元产生一个新子句。这一切即使在理由从不被消费时也会运行 (`reason_bounds_out == nullptr`; 只有最后的压入被门控, :5885)。随后 `apply_bound` 在每次收紧时拷贝合并后的子句 (`auto reason = arena.get(...)`, :5886, 5926——是 `auto` 而非 `auto&`)。SCIP/HiGHS 在域轨迹中仅为每个界存储一个理由指针。每次调用传播器还会分配 ~ 7 个大小为 n/m 的 `vector` 与 `deque` (:5786–5794), 并支付 2 次时钟读取加 8 个原子 `RMW` 性能计数器, 且始终开启 (:35–44、:5784、:5860–5866)。

8.3 冲突池: 没有监视文字

发现 13 (冲突传播将整组子句重扫至不动点; 去重忽略了已存储的哈希). `ConflictPool::propagate_impl` (`bc_pools.hpp:443--541, V: :452–458`) 以 `while(changed)` 循环遍历所有子句——每个不动点轮次都是一次 $O(P \cdot L)$ 扫描, 且每次扫描都为每个子句堆分配一个 `reason` 向量; `has_conflict` 是每个节点一次的完整 $O(P \cdot L)$ 扫描 (:402–419)。`add()` 先做规范化 ($O(L \log L)$), 再针对最近 `max_scan = 512` 个子句运行有界冗余检查 (良好的缓解措施: `scan_begin`, :187–190), 随后对整个池做无界的精确重复扫描, 逐元素比较哈希

与文字向量 (:295–308)，然后进行 *FIFO* 驱逐 `clauses_.erase(clauses_.begin())` (:317)——这是对拥有堆内存的子句的一次 $O(P)$ *memmove*，且没有基于活跃度/*LBD* 的子句老化机制（自 *MiniSat* 以来的 *SAT* 求解器标准做法）。没有变量 $\rightarrow$ 子句索引，没有监视文字：最先进的冲突分析在每次界变更时只需 $O(\text{touched clauses})$ 。并行版 *SharedConflictPool* 增加了一个无锁的 16384 位、 $k=3$ 的 *Bloom* 预过滤器 (`bc_threading.hpp:368--491`)——好主意——但子句驱逐时该过滤器从不清空，因此假阳性率单调增长，并会静默丢弃真正新颖的子句 (:436–439 注释中的 $\sim 0.5\%$ 假阳性数字是干净过滤器下的估计)。

8.4 蕴含图与变量界表

BinaryImplicationGraph: 以 `std::vector<std::vector<Implication>>` 表示 $2n$ 个文字上的邻接表 (`bc_utils.hpp:3001`); `add_implication` 通过在触发桶内线性扫描来去重 (`bc_utils.cpp:5299--5304`)——在 250 000 条蕴含的上限下存在 $O(E^2)$ 最坏情况构建风险 (`bc_implied_bounds.hpp:131--133`); `propagate` 是每次文字最多触发一次 BFS，但会分配两个大小为 n 的 `seen` 数组，且每次调用都通过扫描全部 n 个变量来播种 (:5319–5344)，而它在 `NodeQueue::prune_by_implications` 中对每个排队节点都会被调用 (`bc_utils.hpp:3140`)——复杂度为 $O(\#nodes \cdot (n+E))$ ，且没有基于实际界变更的增量播种。*VariableBoundTable*: 逐列条目列表，对整数目标做互补 MIR 强化 ($f_0 \in [0.005, 0.995]$, `bc_utils.cpp:5412--5433`) 并支持按触发替换 (:5459–5468)。隐含整数检测运行 ≤ 8 趟的不动点 (`bc_implied_bounds.cpp:56`); 隐含列界运行 ≤ 16 趟对所有行的完整扫描，且没有工作列表 (:554–568，跳过非零元数 > 512 的行); 逐条目的活动度移除为每行 $O(k^2)$ (:514–548)。

8.5 目标传播

纯头文件实现 (`bc_objective_propagation.hpp`, 2368 行): 项 (term) 即带成本的变量; 由 $\leq$ 行与团表构造“至少一个有利”的团划分; 由行得到隐含贡献事件 (上限: `kMaxRowNnz=32`, `kMaxRowLiterals=24`, `kMaxEvents=100 000`, :558–560)，其中每个目标的成对事件为 $O(L^2)$ (:764–772); 单文字事件获得互补 MIR 强化。`compute_activity`: 按符号取 $\text{act} = \sum_j c_j \cdot (\ell_j \text{ or } u_j)$ ，并施加团划分修正 $\text{nonfav_sum} - \max(0, \text{best_replacement_delta})$ (:1035–1037)。`propagate`: 最多 4 轮，每轮重新计算完整活动度并线性扫描所有事件 (:1792–1833); 若 $\text{act} > \text{cutoff}$ 则剪枝 (:1847); 逐项截断收紧 $x_j \leq (\text{cutoff} - (\text{act} - \text{contrib}_j))/c_j$ (:2229–2287)。`implied_events_by_literal` 索引被构建 (:757,893) 但从未被 `propagate()` 使用——它恰好只在一个专门位置被引用 (`bac.cpp:32071--32115`); 事件驱动的触发本可将复杂度降为 $O(\text{touched events})$ 。

9 并行化

9.1 设计

状态包 `ConcurrentTreeState` (`bc_concurrent_tree_state.hpp:22--78`) : `SharedCutPool` (`shared_mutex`)、`SharedConflictPool` (`shared_mutex` 加 *Bloom* 过滤器)、`SharedSolutionPool`、`SharedIncumbent`、`ThreadSafeNodeQueue`、`CutRequestQueue`

(容量 `cut_worker_queue_size{64}`)、`AtomicBCStats` (热点计数器正确地做了缓存行隔离, `bc_threading.hpp:1148--1151`)、`WorkStealingPool` (按工作线程的互斥锁保护双端队列, 容量 `work_steal_deque_size{256}`)、`SharedImplicationGraph` (仅追加的弧向量加代际计数器, 带无锁读快路径——干净的设计——但 `publish` 存储批次时不去重也不驱逐, `:1222–1227`: 无界增长), 以及一个 `DeterministicTurnToken`。

角色 (`bc_threading.hpp:240--244`): `Diver` (DFS 加启发式)、`Prover` (最佳界加深度 $\leq \text{max_plunge_depth}\{10\}$ 处的可靠性分支)、`IPMDiver` (当 $m > \text{hybrid_ipm_threshold}\{15000\}$ 时用内点法 (IPM) 求解节点 LP)。分配方式: 混合模式下 `tid 0` 为 `IPMDiver`, 最后一个线程为 `Prover`, 其余均为 `Diver` (`bac.cpp:35717--35720`)。

爬坡启动: 没有。根 LP、割平面与所有根启发式均串行运行; 线程仅在进入搜索树时启动 (`bac.cpp:35655--35750`), 此时共享队列中只有一个根节点; 提前启动的重叠被硬性禁用 (`const bool allow_concurrent_tree = false;`, `:18025–18026`)。启动时, $n-1$ 个探索线程以 10ms 的条件变量超时对单节点队列空转轮询, 直到首次分支产生子节点; 每次弹出失败时, 任何解池规模 ≥ 2 的探索线程都会运行一个不受限流的 $O(\text{nnz})$ 交叉启发式 (`bc_parallel.cpp:1086--1113`) ——在工作饥饿状态下, k 个线程以 100Hz 空转并执行完整 LP 检查。

工作分配: `Diver` 将优选子节点压入本地双端队列, 并把兄弟节点溢出到全局队列 (`bc_parallel.cpp:1505--1517`); 窃取时从随机受害者处取走最旧的 (队首) 节点; 获得现任解后, 本地双端队列在每次循环迭代都被排空到全局最佳界前沿 (`drain_local_dfs_to_proof_frontier`, `:1021–1048`)。

发现 14 (每个工作线程每次迭代都在全局互斥锁下进行 $O(Q)$ 队列扫描)。获得现任解后, `drain_local_dfs_to_proof_frontier` 调用 `node_queue.prune_by_incumbent(...)`——在全局锁下做一次完整的 $O(Q)$ 扫描, 并对每个被剪节点从四个索引结构中逐一删除 (`bc_threading.hpp:1038--1057`)——这发生在每次探索循环迭代 (`bc_parallel.cpp:1048`), 并在每次子节点求解后再次发生 (`:1427`)。再叠加每次弹出的 $O(|\text{dfs}|)$ 删除 (发现 10), 一次求解过程中的队列维护总开销为 $O(Q^2)$, 且全部串行化在那把每个工作线程执行 `push/pop/lower_bound` 都要经过的互斥锁上。最先进的设计在弹出时惰性剪枝 (该代码同样做了惰性剪枝, `bc_parallel.cpp:1116`——这次扫描与其自身的惰性检查是冗余的)。1ms 监控线程对同一批锁持续施压 (`bac.cpp:35766--35796`: 每趟都要锁全局队列、每个工作窃取双端队列以及活跃界互斥锁)。

现任解广播: 先对目标值做 CAS, 再在互斥锁保护下拷贝 x (`SharedIncumbent::try_update`, `bc_threading.hpp:540--551`); 读取方使用避免撕裂读的 `snapshot()` (`:577–584`)。

发现 15 (`SharedIncumbent` 先发布目标值再发布解向量)。对 `obj` 的 CAS 先于互斥锁保护下的 $x = \text{new_}x$ (`bc_threading.hpp:541--548`): 两个并发更新者可能交错执行, 使得存储的 x 与存储的 `obj` 分属不同的目标值——解池随后会保存不匹配的 (x, obj) 对 (`bc_parallel.cpp:1107--1109`)。仅限启发式一侧 (对证明的正确性没有影响), 但这确实是一个发生在其注释声称保持一致性的结构中的真实竞态。

伪成本同步: 一把全局 `pc_mtx` 守护一个稠密的全列 `std::vector<PseudoCost>` (`bc_concurrent_tree_state.hpp:34--35`) ——紧凑的仅含可分支列的表是存在的 (`CompactPseudoCostTable`, `bc_types.hpp:578--628`) , 但只在遗留的顺序路径中使用。

Prover 每个节点都在锁下深拷贝整个表，在本地探测，然后在同一把锁下按所有可分支索引合并回去 (bc_parallel.cpp:1198--1269) ——形成了一种设计内生的护送 (convoy) 效应：Diver 为读取分数值 (:1276–1280) 和进行逐节点 PCUpdate 合并 (:1454–1461) 也要获取同一把互斥锁。

割平面共享：探索线程把 CutRequest 发送给唯一的割平面工作线程 (bc_parallel.cpp:966--979)；该工作线程仅为深度 0 的请求生成单纯形表割，且只把根割导出到共享池（节点受限的 Gomory 割“在共享池中将是不健全的”，:1595–1640）——默认仅有根节点流量。每个 CutRequest 都要移动一整个 SimplexResult——CSC+CSR 矩阵以及稠密的 Eigen::MatrixXd basis_inverse (dual_simplex.hpp:315--327; 在 $m \leq 4096$ 时物化, dual_simplex.cpp:1603--1617) ——外加三个稠密向量，而拷贝发生在队列容量检查之前，该检查又经常丢弃请求 (bc_parallel.cpp:968--978, cut_requests_dropped 计数器)。

冲突/蕴含共享：总开关 share_conflict_learning_across_threads{false}——刻意关闭 (options.hpp:228, 注释警告存在跨线程作用域错误)；蕴含共享开启 (share_implications_across_threads{true}, 批量 8)。

确定性：默认 deterministic_parallel{false} (options.hpp:334)；即使开启，弹出与压入之间的 LP 求解仍存在竞态，因此现任解发现的时机——以及随之而来的剪枝、伪成本合并顺序（浮点加法不具结合律）与搜索树形状——都不是完全确定的。

10 终止与间隙处理

相对间隙： $(z_{\text{inc}} - z_{\text{lb}}) / \max(1, |z_{\text{inc}}|)$ (bac.cpp:33956--33957)。关键的默认值是 require_tree_exhaustion_certificate{true} (options.hpp:253--258)：每个 rel_gap ≤ gap_tol 出口都被 !opt.require_tree_exhaustion_certificate 门控 (V: bac.cpp:33970--33973；相同的守卫还出现在:34209–34213 与:35600–35605)，使得 rel_gap ≤ 1e-9 成为唯一基于间隙的停止条件 (:33968–33969)：

```
const bool proven_optimal_exact = incumbent_proof_valid && rel_gap <= 1e-9;
if (proven_optimal_exact ||
    (root_gap_certificate_ready && incumbent_proof_valid &&
     !opt.require_tree_exhaustion_certificate &&
     rel_gap <= opt.gap_tol)) { ... } // bac.cpp:33968-33973
```

默认情况下，求解器实际上必须耗尽整棵树（或证明界已与现任解吻合至 10^{-9} ）；面向用户的 gap_tol 几乎是失效的——Gurobi/SCIP/HiGHS 都会在 MIP 间隙处停止。相关：incumbent_quality_reject_factor{1e6} (options.hpp:169--175) 允许接受劣化至对偶界 $10^6 \times$ 的启发式现任解，而证明尾部模式（纯最佳界排空）由任何证明有效的现任解触发 (bac.cpp:23468--23477)。**树重启：没有**——节点循环中任何地方都不存在重启机制；重启热启动状态会显式抛出异常（“割池与继承界的重启状态尚未实现”，bac.cpp:36788--36796）。从现任解出发、带着学到的割平面/伪成本重启，是 SCIP/HiGHS 中将树规模缩小 1.5–2× 的标准手段。间隙停滞停止条件存在但被禁用 (stall_node_window{0}, options.hpp:560--564)。

11 综合严苛审计

本节对结构性发现进行分级排序。所引用的各节包含证据；严重度反映其对大规模实例性能以及结果可信度的影响。

11.1 严重度 1 ——与最先进水平的结构性性能差距

发现 16 (每节点完整稠密域副本；基于差分的节点设计已存在，却是死代码). *Node* (*bc_types.hpp:330--388*, *V*) 拥有四个稠密 n 向量 (*lb*、*ub*、*x_relax*、*x_seed*)，外加十个 *std::vector* 载荷 (域轨迹、原因界、各自持有一个稀疏向量的局部割、以“向量之向量”表示的局部冲突子句, ...)。*branch_child()* (*:360-387*, *V*) 复制 *lb/ub* (每个均为 $O(n)$ 稠密)，并深拷贝全部十个载荷——包括持有堆内存 *Eigen::SparseVector* 的 *local_cuts*——每次分支执行两次 (*bac.cpp:21208--21231*, *35222--35231*; *bc_parallel.cpp:1339*)。(它并不复制 *x_relax/x_seed*——*:359* 处的注释写明了这一点；载荷复制才是问题所在。) 不计载荷，开放节点集的内存在即为 $\Theta(N_{\text{open}} \cdot n)$ ：当 $n = 10^4$ 时，仅 *lb+ub* 每个节点就耗费 $\approx 160\text{KB}$ —— 10^4 个开放节点即 $\sim 1.6\text{GB}$ 。*SCIP* (自 3.0 起) 与 *HiGHS* 每个节点仅存储 $O(\text{depth})$ 个界变更差分，并通过沿路径回溯重新推导域。本仓库包含的恰恰就是这种设计——*TreeNode + TreeDomainManager + TreeNodeArena* (*bc_types.hpp:147--314*) ——而全仓库范围的 *grep* 显示零使用 (*V*)。代码用如下注释绕开了它自己的设计：“The popped parent is dead after branching. Move its $O(n)$ domain and proof state into one sibling instead of copying both *lb* and *ub* again” (*bac.cpp:35227--35228*)。

发现 17 (每个节点 *LP* 都新建 *Highs*、完整序列化并冷因子分解). 发现 4: 每个节点的 $O(n+m+\text{nnz})$ 序列化、*HighsLp* 构造以及完整 *INVERT*——这正是 *SCIP/SoPlex*、*HiGHS-MIP* 与 *Gurobi* 用一个持久 *LP* 对象 (在保留的因子分解上接收增量界变更) 所消除的成本。增量机制已经建成 (*resolve_same_structure*、*update_standard_form_bounds*、根节点实时追加)，却在每个搜索树调用点被关闭 (*bc_legacy_helpers.cpp:203--205*)。每次求解还有额外开销： $O(m^3)$ 的稠密基求逆、向每个 *SimplexResult* 复制完整标准形的 *out.form = sf* (*dual_simplex.cpp:1406*)、为证明再复制一份标准形 (*bac.cpp:31105--31106*)，以及重复的 $O(\text{nnz})$ 残差审计。峰值时，约束矩阵同时以 *LPMoel-CSC + HiGHS-CSR* 镜像 + *SF-CSC + SF-CSR* ($\times T$ 个线程工作区, *bc_types.hpp:782--784*) + *BCDomain* 行主序副本 (*bc_domain.hpp:93--94*) + *HiGHS* 内部结构 + *A_owned_* (*bc_relaxation.cpp:289*) 的形式存在—— nnz 的 $\gtrsim (4 + 2T)$ 份活跃副本。

发现 18 (尽管已知变更 API 沦为死代码，*LP* 热路径上每个子节点仍做 $O(n)$ 簿记). *update_standard_form_bounds* 为检测变更而扫描全部 n 列，并在每个子节点、每次探测时重写全部 n 个 *var_ub* 条目 (*dual_simplex_api.cpp:873--959*；调用于 *bac.cpp:30719*, *35020*)；接收已知变更的变体 (*update_standard_form_bounds_incremental*, *:976*) 调用点为零 (*V*)。在 $5 \cdot 10^4$ 个节点 $\times n = 10^5$ 下，这相当于 $\sim 10^{10}$ 次纯簿记的标量操作。

发现 19 (冲突学习无监视文字；割池无哈希). 发现 13 与 9: 每个不动点轮次做 $O(P \cdot L)$ 全扫描单元传播，每次去重做全池扫描，*FIFO* 淘汰，失效的老化机制，以及一个误报率单调增长的 *Bloom* 过滤器。这些都位于每节点路径上 (每次弹出节点都做冲突检查, *bac.cpp:34219--34222*)。

发现 20 (默认终止条件要求近乎精确地穷尽搜索树; `gap_tol` 形同虚设). 节 10. 对 MILP 求解器而言, 这是一阶的可用性/性能缺陷: 在那些间隙的最后 0.01% 要耗费 90% 搜索树的实例上 (常见情形), 任何竞品都会在 `gap_tol` 处停止, 而默认配置却会一路磨到时间/节点上限。

11.2 严重度 2 ——热路径上的算法性浪费

发现 21 (强分支: 每次探测都复制完整标准形并运行完整单纯形求解). `probe_sf = tree_base_sf`; (`bac.cpp:35019, V`) 复制两种方向的矩阵布局; 随后每个方向都在一个全新的 HiGHS 上运行一次完整单纯形求解 (`bac.cpp:35027--35029`) (§5.3), 既没有针对探测的迭代上限, 也没有对偶界截断。被丢弃的探测只换来伪成本样本。每次可靠性决策的成本: $O(\text{candidates} \times (\text{nnz} + \text{serialization} + \text{cold factorization}))$ 。

发现 22 (每节点重复的 $O(\#\text{integers})$ 分数性扫描). `fractional_branchable_indices` 在弹出节点时运行 (`bac.cpp:34410`), 每个子节点运行 (:30794), 在预队列刷新中运行 (:33785); `compute_node_estimate` 每个子节点还要再扫描所有可分支变量两次 (:35472–35480); `RINS`/交叉检查各自再增加三次 $O(n)$ 遍历 (:35524–35566)。代码并未维护增量分数集合 (HiGHS 在其域中增量维护 `fractionalints`)。

发现 23 (根探测对同一固定值重复传播两次, 并付出 $O(n_{\text{bin}} \cdot (\text{nnz} + n))$ 的代价). 对多达 1000 个未固定二元变量中的每一个: 先对 $x=0$ 做完整传播 (`bac.cpp:22233`), 再对 $x=1$ 做完整传播 (:22248), 然后在一份全新副本上再次对 $x=0$ 传播 (:22275–22278), 每次传播后都跟一次 $O(n)$ 的蕴含提取扫描 (:22260–22292)。SCIP/HiGHS 会存储第一遍探测的结果。

发现 24 (所有模型行分离器对每行分配稠密 $O(n)$ 向量). `add_basis_mir_cuts` 每行分配四个零初始化的稠密 n 向量 (`bc_cuts.cpp:5495, 5511, 5534, 5563`); `add_row_mir_cuts` 再分配四个 (:6079, 6090, 6106, 6123); `cover`、`flow-cover`、`implied-bound`、`zero-half`、`clique` 各自为每个候选构建稠密 n 向量——每个割平面族每轮产生 $O(m \cdot n)$ 的 `memset` 流量 (在 $m = 12000$ 、 $n = 14000$ 时每轮即达数 GB 的零写入), 而这发生在任何实际工作之前; 此外还有按族完整的行主序矩阵拷贝 (:5321, 5490–5491, 5676, 5810, 5845, 5970, 6075–6076, 7770, 7936) ——每轮 `All` 至少 ≥ 6 次完整 $O(\text{nnz})$ 转换, 这还不包括 `add_rows_to_lp` 按族批次对整个矩阵的 `triplet` 重建 (§6.8)。最先进的分离器严格在稀疏行表示上操作。

发现 25 (传播原因追踪存在平方级过度工程, 且始终开启). 发现 12: $O(\text{nnz}_{\text{row}}^2)$ 的 `id` 收集、按非零元的子句物化、`eq` 路径上按非零元的堆分配, 以及无界的 `arena` 增长——即使从未请求过原因。

发现 26 (传播热路径上易相消的活动度算术). 无处不在的模式 `residual = rhs - (min_activity - contrib)` (`bc_utils.cpp:5032, 6131, 6198, 6234`; 预求解 `milp_presolve.cpp:134, 181, 201, 454`) 每次访问都从零重算行活动度, 并且在行活动度较大时减去两个几乎相等的量——这是数值上最弱的表述形式, 却位于最热的数值路径上。SCIP 维护增量活动度, 并以受控方式重算。代码通篇使用未经缩放的固定绝对容差 (10^{-9} 的界改进阈值、 10^{-15} 的系数截断、`bounds_consistent` 中的 10^{-12})。

发现 27 (每个获得现任解后的节点 LP 都默认开启对偶证明构造). `enable_reduced_cost_proof_cut_resolve{true}` (`options.hpp:311`) 使每个获得现任解后的节点都运行 `build_full_dual_proof_row + build_reduced_cost_mass_dual_proof_row` (`bac.cpp:31025,31063`): ~ 3 次完整稀疏矩阵向量乘、两次手工实现的 $O(\text{nnz}) A^T y$ 遍历 (`bc_utils.cpp:1775--1790,1850--1867`)、一次 `BTRAN`, 以及 ~ 6 个稠密 $O(n)$ 临时量——证书机制位于关键路径上, 而商业求解器仅在真正分析冲突时才惰性运行它。这些解析例程每次审计还增加 $O(T^2)$ 的轨迹重放不动点闭包 (:2414-2439, 3454-3477), 以及按候选复制并重新规范化整个子句的贪心选择循环 (:4573-4684)。

发现 28 (监视循环与队列扫描使并行证明阶段串行化). 发现 14; 此外还有一个不使用条件变量的 `1ms` 轮询监视器 (`bac.cpp:35766--35767`), 它白白烧掉一个核, 并为每个终止事件增加至多 `1ms` 的延迟。

11.3 严重度 3 ——正确性边缘问题与有效性缺口

发现 29 (隐含界割平面族要么空泛要么重复). 发现 6 (代数过程已验证)。当 10^{-7} 准入检验拒绝它时无害; 当噪声使其通过时, 则成为一经过缩放的重复行。

发现 30 (`Flow-cover` 存在已被承认的有效性缺口, 仅保留为诊断用途). 该族自己的注释 (`bc_cuts.cpp:8328--8332`) 声称它在预求解或 `ranged` (双边) 行上“*can produce rows that cut off the true integer frontier*”。它仍可通过 `CutType::FlowCover` 到达——对任何期望生产行为而选择它的调用者而言, 这是一个陷阱。

发现 31 (受扰 LP 的最优解被接受为节点结果; 基于字符串匹配的失败分类法). `L1` 回退在不做任何对账的情况下接受右端项受扰 LP 的最优解作为有效节点界 (`bc_fallback.hpp:273--284`); 失败分类依赖对人类可读状态字符串的子串匹配 (:78-97)。来自受扰 LP 的节点界并不是未受扰节点的认证界——这与除此之外严格的 *Farkas* 证书纪律相抵触 (§5.6)。

发现 32 (预求解可能静默丢弃已证明的不可行性). 节 4.2: 双重不可行探测与不可行空行都不产生任何状态; `PresolveStats` 没有不可行标志。一个可证明不可行的实例仍会进入 `LP/B&C`。

发现 33 (无界一般整数变量未经检查即进入搜索树). 节 2.2: 直接进入路径上没有对自由整数的界健全性检查; 对自由整数分支会产生无穷的子节点域, 并可能静默地永不终止 (只有全局限制能阻止运行)。

发现 34 (哈希量化冲突静默合并不同的割/文字). 割签名将系数量化到 10^{-4} 、将文字值量化到 10^{-6} (`bc_pools.hpp:30--35,92--100`) ; 域签名将界量化为字节串 (`bc_utils.hpp:1945--1986`)。冲突被当作重复处理——去重逻辑中一条 (低概率的) 静默错误结果通道。

11.4 严重度 4 ——如今已损耗性能或信任的工程债

发现 35 (一个 35000 行的单一函数就是整个引擎). 节 3.1: ~ 60 个按引用捕获的 `lambda` 共享数百个可变局部变量; 正确性不变量 (例如“*keeps the old basis/ x_{relax} from being used with a stronger domain than the LP that produced it*”, `bac.cpp:34247--34249`) 靠跨纪元标志 (`domain_`

`learning_epoch` 等, `bc_types.hpp:351--354`) 的约定来维护。这正是上文编目的防御性重扫描与重复制的根本原因: 没有任何组件可以被孤立地测试、复用或推理。

发现 36 (大规模的死代码与分叉代码). 已验证为死代码: `search/ + parallel/ + root/` 模块 (~1400 行, 参与编译, 调用者为零); `TreeNode/ TreeDomainManager` 差分域设计 (`bc_types.hpp:147--314`); 经典 GMI 分离器与一个 MIR 变体 (~600 行, `bc_cuts.cpp:120--301, 303--466, 6193--6350`); `detail::milp_presolve` (~620 行, `bc_utils.cpp:97--716`); `presolve_inplace` (`milp_presolve.cpp:1499--1667`); `node_bound_propagation_tracked` (仅测试使用); `update_standard_form_bounds_incremental`; `age_and_purge`; 被宣传的树 RINS 子 MIP (注释与现实不符, `bac.cpp:35549--35551`); 默认选项下的割工作线程; 孤儿头文件 `bc/cuts/cut_types.hpp` 与 `bc/branching/pseudocost.hpp`——前者在同一命名空间中定义了一个 11 族的 `CutFamily` 枚举和一个 2 字段 `PoolCut`, 与现役的 7 族枚举和 9 字段 `PoolCut` (`bc_types.hpp:641--650, 317--327`) 冲突, 首次被包含即构成 ODR 违反。这里的死代码并非无害: 它已经分叉 (角色分配、可靠性规则、`PCUpdate` 字段名), 并会误导任何读者对求解器实际行为的理解。

发现 37 (`BCOptions` 是无法管理的扁平团块, 含 ~230 个公共字段). `options.hpp:86--870`: 分节仅以注释形式存在; 它暴露了内部递归标志 (`_is_stage3_submip`, :784)、提交信息式的调参理由 (“*P7.6: Raised from 5000→15000*”, :350–353)、作为通用 MILP 求解器默认值硬编码的 `UC/SCUC` 实例类别阈值 (`scuc_min_binaries{500}` 等, :824–837), 以及 ≥ 30 个仅转发给内嵌 `HiGHS` 的旋钮。最先进的求解器只公开其中一小部分, 并将实例类别调参与公共契约隔离。一个隐藏的环境变量通道 (“*200+ environment variables*”, `bc_env_options.hpp:7--9`; 以及零散的 `std::getenv`, 如 `bc_legacy_helpers.cpp:42--67`, `bac.cpp:1708, 2407--2410`) 使运行结果无法仅凭 `BCOptions` 复现; 像 `MIPSOLVERS_ROOT_SEPARATION_ONLY` 这样影响算法的开关会不可见地改变行为。

发现 38 (默认非确定性). `num_threads{-1}` → 硬件并发, 配合工作窃取, 且 `deterministic_parallel{false}`; 该选项自己的注释称弹出顺序随后取决于“*OS scheduling jitter*”, 并且撕裂的现任解读取可能导致“*different prune/keep decisions*”(`options.hpp:314--334`)。位级可复现性需要手动显式启用——即便如此也未完全实现 (§9)。

11.5 做得好的地方 (为平衡起见)

缓存行隔离的热原子操作 (`bc_threading.hpp:1148--1151`); 经由 `try_share_indices_from` 的基下标共享 (`dual_simplex.hpp:258--270`); 标准形层的增量界更新 (`dual_simplex_api.cpp:899--920`) 与根节点实时追加事务 (`bc_relaxation.cpp:1121--1274`)——结构上正确的机制, 只是没有用在每节点路径上; 对超线性割池扫描施加的扫描后缀上界 (`bc_pools.hpp:187--190`); `SharedImplicationGraph` 中基于世代计数器的无锁读取路径 (`bc_threading.hpp:1233--1253`); 以 Farkas 证书为门槛的剪枝, 数值失败时绝不剪枝 (`bc_fallback.hpp:82--84, 232`); 按族以滑动平均进行的割效能门控 (`bc_types.hpp:655--705`); `cMIR` 流水线中的扩展精度聚合 (`bc_cuts.cpp:4232--4335`); 强分支探测 LP 复用为子节点 LP (`bac.cpp:30902--30909`); 基于域签名的重复节点抑制 (代价昂贵, 但确是真正的算法思想); 割/启发式武器库的整体广度, 其覆盖范围确实接近 `HiGHS`。

12 占位符与虚假复杂性普查

审阅此代码库的人会注意到，其中相当一部分看起来像求解器机制，却什么都不做。本节用可复现的普查量化这种印象，然后对定性发现进行分类。以下一切内容均可用所给出的 `grep` 独立核验。

12.1 普查方法（可复现）

- **普查 A（选项）**：对 `options.hpp` 中可通过声明模式 `grep` 提取的全部 230 个字段声明，按成员访问读取——即 `.<field>` 或 `-><field>` 的匹配——在 `src/`、`include/`、`benchmark/` 与 `tests/` 范围内计数。
- **普查 B（统计）**：对 `BCStats` (`bc/stats.hpp`) 的 370 个字段声明做同样的统计。
- **普查 C（隐藏配置）**：原生 MILP 代码树中传给 `getenv(...)` 的不同字符串字面量，及其调用点计数。
- **普查 D（自我标记）**：针对 `TODO/FIXME/HACK/XXX`、`scaffold` 以及 `not implemented` 式自我承认的 `grep`。

12.2 定量结果

发现 39 (230 个公共选项字段中有 9 个从未在任何地方被读取)。普查 A 对 230 个字段中的恰好 9 个返回零读取（生产代码、基准与测试皆然）(V)：`highs_style_root_separation`、`highs_style_branch_scoring`、`highs_style_reliability_probing` (`options.hpp:449,451,453`——三个被宣传的“HiGHS 对齐基准模式”，却不切换任何行为)、`cglp_normalization` (:518——CGLP 构建器硬编码了单一规范化方式,`bc_cglp_model.cpp:171--179`)、`cglp_register_in_cut_pool` (:532——其自己的注释承认 “[Scaffold — Phase 4 global dedup, NOT YET IMPLEMENTED]”，:519)、`huge_problem_n_threshold` (:817)、`large_uc_force_ipm_row_threshold` 与 `large_uc_force_ipm_bin_threshold` (:826–827)，以及 `simplex_tol_floor` (:861)。此外，230 个字段中有 71 个恰好仅被读取一次。每一个从未被读取的旋钮都是对用户的虚假承诺：设置它改变不了任何事。

发现 40 (370 个统计字段中有 19 个从未被引用；一整个遥测族处于惰性)。普查 B 对 370 个 `BCStats` 字段中的 19 个返回零引用 (V)，包括整个 8 字段的 `rowdual_cmir_*` 簇 (`..._added`、`..._bound_moves`、`..._candidates`、`..._lift_max`、`..._lift_sum`、`..._rejected`、`..._target_coeff`、`..._violated`) 以及 `sub_root_*` 镜像簇——为从不写入任何内容的功能所声明的遥测。75 个字段至多被引用一次。再结合 `BCStats` 被按值复制进每个结果这一发现，统计界面实质上是一场只写的审计表演。

发现 41 (由 ≥ 51 个环境变量构成的隐藏配置通道，且零自我标记)。普查 C 发现原生 MILP 代码树中有 51 个不同的环境变量通过直接的 `getenv("...")` 字面量被读取 (V)，仅 `bac.cpp` 一个文件就有 125 个 `getenv` 调用点 (V)；其自身的基础设施注释声称有“200+ environment variables”(`bc_env_options.hpp:7--9`——可用 `grep` 核验的数量是 51；该声称按原文无法核验)。普查 D 在整个原生 MILP 代码树中发现零个 `TODO/FIXME/HACK/XXX` 标记 (V)——本文记

录的占位符没有一个带有任何自我承认——同时还有 17 个 $P-x.y$ 调参计划标记 ($P2.1 \dots P9.2$) 与 *Phase 1--4* 脚手架注释, 以及 36 处 `[[maybe_unused]]`。

12.3 占位符层的定性分类

发现 42 (形同陷阱的公共 API). 公开声明并写有文档、却无条件抛异常的入口点: `BCCallbacks::node_selector` 与 `cut_selector` (`ml_hooks.hpp:77--94`; 抛异常位于 `bac.cpp:36850--36853`) ; `BCWarmStart` 的 `dual-row/dual-col/basis/cut-pool/inherited-bound` 字段——“*reserved for a future restart format*” (`warmstart.hpp:9--11`; 抛异常位于 `bac.cpp:36788--36797`) ——包括定义完整却无法使用的类型 `BCSimplexBasis` 与 `BCCutRecord` (`warmstart.hpp:33--53`) ; `StrategyDispatcher::analyze` 返回一个全零的 `ProblemCharacteristics`, 并带有一个“*Phase 3b*”TODO (`dispatcher.cpp:226--236`), 使求解器“选择”沦为硬编码的名字列表 (`dispatcher.cpp:66--86`)。

发现 43 (宣传了却不存在的能力与虚假文档). 孤儿头文件 `bc/cuts/cut_types.hpp` (无任何地方包含) 宣传了 11 个割平面族——包括 *StrongCG*、*Knapsack*、*GCD*、*DisjunctiveCuts*——它们在实现中无一存在, 且其定义会与现役定义发生 ODR 冲突 (§6)。孤儿头文件 `bc/branching/pseudocost.hpp:18` 声明了第二个 `detail::PseudoCost`, 其方法 `down_lcb()`、`up_lcb()`、`combined_score()` 只有声明、从未在任何地方定义 (:62-69) ——调用其中任何一个都是链接错误。与代码矛盾的文档: `bc/api.hpp:16--17` 声称原生对偶单纯形是默认 LP 引擎 (默认实为内嵌 *HiGHS* 内核, `options.hpp:135--137`) ; `milp_presolve.hpp:16--17` 宣传了并不存在的“*row/column nonzero counts for $O(1)$ singleton detection*” (`milp_presolve.cpp:309--318, 382--391` 为重新扫描) ; 双元素行替换的注释称“*prefer the one with more constraint appearances*”, 而代码实际偏好出现次数更少者 (`milp_presolve.cpp:501--502` 对比:517-527) ; 名为 *StrictHiGHS* 的适配器并不运行 *HiGHS* 的 MIP 求解器 (§3.2) ; 注释中所谓保留“*the full RINS sub-MIP for the near-termination gate*”描述的是不存在的代码 (`bac.cpp:35549--35551`)。

发现 44 (会执行却不计算任何结果的机制). 按构造就在热路径上运行、却不产生任何效果的代码: 隐含界割平面族重新推导出它自己的源行, 且仅靠数值噪声才能将其准入 (发现 6) ; `augment_variable_bound_table_from_cut_rows` 为每条割行的每个整数元素计算 *floor/ceil* 域收紧, 然后只是递增一个统计量——收紧后的值从未被应用 (`bc_implied_bounds.cpp:849--865, :1012-1028` 处完全相同) ; `implied_events_by_literal` 索引被构建 (`bc_objective_propagation.hpp:757, 893`), 却从未被 `propagate()` 使用, 后者改为扫描全部事件 (:1795) ; 割池“老化”: 每个割出生时年龄即为 45/50 (`bc_pools.hpp:595`), 而 `age_and_purge` 从未被调用 (§6.8), 于是精巧的年龄/淘汰机制退化为盲目 FIFO; `incumbent_quality_reject_factor{1e6}` (`options.hpp:169--175`) 是一道“质量门”, 它接受目标值高达对偶界 $10^6 \times$ 的现任解——一道什么也拦不住的门。

发现 45 (完全建成却在最后一寸被关闭的机制). “实现了、接好了、然后被废掉”的模式反复出现: 持久 LP 节点求解 (发现 4) ; `update_standard_form_bounds_incremental` (调用点为零, §5.2) ; 早期并发树启动 (`allow_concurrent_tree = false` 硬编码, `bac.`

`cpp:18025--18026`); 割工作线程 (`use_cut_worker = ...` ~~且~~ `max_cut_depth > 0` 而 `max_cut_depth{0}`, `bac.cpp:18096`, `options.hpp:101`); `deterministic_parallel` 即使启用也不能使运行确定 (§9); `RootSolvePhase` (`root/root_solve.cpp`) ——死代码, 且若真被复活, 将以 `simplex = nullptr` 运行一条降级的根流水线 (`root_solve.cpp:122--128`)。

发现 46 (内部已深度腐烂的死模块). 编译了却不可达的 `search//parallel//root/` 栈 (§3.1) 不仅仅是死代码——它的内部不一致恰恰证明它从未被执行或测试过: `SequentialSearchDriver` 从不应用 `NodeEvaluator` 构建的 `PCUpdate` (所有截断/推理观测都被静默丢弃); `BCSolveContext::update_dual_bound` 从未被调用, 因此 `gap()` 永久为无穷, 驱动器的间隙终止检查永远无法触发 (`parallel/shared_state.cpp:56--63`, `search/sequential_search.cpp:243--246`); 串行驱动器却持有一把 `pseudocost_mutex` (`search/sequential_search.cpp:80--83`) ——虚假的线程安全; 渐进舍入构建了 `round_opt`, 随即用 `(void)round_opt` 将其丢弃, 改用默认构造的 `BCOptions{}` 求解 (`search/node_evaluator.cpp:367--378`); 已死的角色调度器 (`parallel/parallel_search.cpp:287--305`) 已与现役实现分叉 (`bac.cpp:35717--35720`)。

12.4 占位符层为何重要

普查用计数坐实了“看起来复杂、实则虚假”的印象: 经直接测量, 公共选项界面的 $\approx 4\%$ 与统计界面的 $\approx 5\%$ 处于惰性, 整个子系统被编译却不可达, 若干公共契约是陷阱。其代价是具体的, 而非审美问题: **(i) 错误归因**——文档与适配器名称描述的是一个并非实际运行的求解器 (对“StrictHiGHS”做基准测试的用户, 实际测的是运行在 HiGHS LP 内核之上的原生搜索树); **(ii) 陷阱界面**——调用者可能选中惰性、抛异常或公认不可靠的选项、回调、热启动以及一个割平面族 (FlowCover); **(iii) 被误导的投入**——调参与评审时间被花在从不执行的机制上, 而分叉的死代码副本向每一位新读者传授错误的算法。为平衡起见: §3.3 的现役流水线是真正可用的——占位符层在很大程度上是包裹在可工作核心之外的附加复杂性, 而这恰恰是它误导人的原因: 它从可工作的部分借来了可信度。

13 优先级排序的建议

按“每单位工程风险所带来的预期影响”排序:

1. **首先清除占位符层** (§12): 删除或接通 9 个从未读取的选项与 19 个从未被引用的统计字段; 移除死代码 `search//parallel//root/` 模块、孤儿头文件, 以及被 `#if 0` 禁用的分离器; 使每个剩余的公开选项要么真正生效、要么不复存在; 把所有环境变量统一经由 `bc_env_options()` 路由, 并在 `BCResult` 中记录生效的集合。这是零风险、纯删除的工作, 且是信任任何后续测量的前提。
2. **翻转终止默认值**: 遵从 `gap_tol` (`require_tree_exhaustion_certificate{false}` 或两级停止)。零风险、首要的可用性修复。
3. **重新启用逐节点的持久 LP 路径** (或移除 `cached_sparse_basis.reset()`, 并将全部十一处基相关调用点经由 `resolve_same_structure` 以安全重绑定的方式路由): 消除逐节

- 点的序列化 + 冷因子分解——很可能是单项最大的墙钟收益。需要一次仔细的兄弟别名（sibling-aliasing）分析（reset 注释暗示曾经做过一次；无论结论如何都应将其记录在案）。
4. **差分域节点**：按节点存储界变更差分（死代码 `TreeNode/TreeDomainManager` 可作为起点）；使 `branch_child()` 达到 $O(\text{changes})$ ；把逐节点的 `x_relax/` 载荷向量改为指向共享结构的引用。
 5. **砍掉隐含界割平面族，或改为从其他行推导**（它目前只是重发其来源行）；把覆盖 DP 提升置于精确算术的门控之后（精确超加性提升器已存在于同一文件中）；修复或移除 `flow-cover`，而不是让一个已知不健全的割平面族保持可选。
 6. **为割池建立哈希索引**；用已存储的哈希做去重分桶；让淘汰感知效能；要么接通 `age_and_purge`，要么删除整套老化机制。
 7. **为冲突池引入监视文字（或变量 → 子句索引）**；基于 LBD/活跃度的子句老化；在淘汰时清空 Bloom 过滤器或为其打上纪元（epoch）标签。
 8. **让节点传播的成因惰性化**：仅当 `reason_bounds_out != nullptr` 时才计算 `ReasonArena` 合并；将等式路径的 `ids` 缓冲区外提（与:6123–6125 相呼应的一行修复）；存储成因指针，而非子句副本。
 9. **在树中使用已知变更的界 API**（`update_standard_form_bounds_incremental`）——分支知道被改变的列； $O(n)$ 扫描与 `var_ub` 全量重写在今天就可避免。
 10. **纯稀疏分离器**：移除逐行的稠密 n 向量与逐族的行主序矩阵副本；跨族、跨轮共享同一个行主序工作区；通过增量式 `addRows` 风格的路径插入割，而非三元组重建。
 11. **在活跃 LP 上做强分支**，带迭代上限与对偶界截断（SCIP 风格），复用节点 LP 对象，而非复制标准形。
 12. **队列重设计**：一个带索引的二叉堆（以界与估计值为键）或 HiGHS 风格的配对堆；去掉 DFS 向量擦除、字符串域签名（改用 64 位滚动哈希或界变更 `trie`），以及全队列范围的 `prune_by_incumbent` 扫描（弹出时的惰性剪除已经存在）。
 13. **让预求解带状态**：把不可行性传播到 `PresolveStats` 并提前中止求解；重建时保留区间行；增量维护活动度（复用 `BCDomain`）；保留由探测导出的蕴涵。
 14. **沿已以 `lambda` 形式存在的接缝分解 `branch_and_cut_lp`**（`process_single_child`、根割循环、启发式），在同一轮中删除死代码 `search//parallel//root/` 栈与孤儿头文件；在此之前，停止编译这些死模块以免混淆。
 15. **可复现性**：把所有环境读取缓存在 `bc_env_options()` 之后；在 `BCResult` 中记录生效选项；提供一个确定性预设（固定线程数、禁止工作窃取、`deterministic_parallel{true}`），并书面记录其限制。

A 默认参数表（经代码核实）

参数	默认值	位置（未注明者均为 <code>options.hpp</code> ）
<code>max_nodes</code>	50 000	:89
<code>max_lp_iter</code>	500	:90
<code>time_limit_sec</code>	120.0	:91

参数	默认值	位置（未注明者均为 options.hpp）
int_tol / gap_tol / lp_tol	10^{-5} / 10^{-4} / 10^{-6}	:92–94
root_cut_rounds / cuts_per_round	10 / 20	:99–100
max_cut_depth	0（仅根节点割）	:101
gmi_min_efficacy / gmi_max_density / gmi_max_parallelism	10^{-3} / 0.35 / 0.90	:102–104
gmi_activity_weight	0.5	:107
cuts	CutType::All	:108
branching	Pseudocost	:113
node_sel	Hybrid	:114
node_estimate_aggregation	Sum	:115–116
use_simplex_lp_nodes / use_ipm_root / use_ipm_nodes	true / false / false	:121–123
ipm_root_probe（+ 迭代数/时间）	true（20 / 5.0s）	:132–134
lp_kernel_backend	LpKernelBackend::HiGHS	:135–137
strict_highs_mip_contract	false	:141
use_feasibility_pump / use_progressive_rounding	true / true	:160–161
use_papilo_presolve / papilo_aggressive	true / false	:162–163
incumbent_quality_reject_factor	10^6	:169–175
enable_feasibility_jump（翻转/修复）	true（512 / 3）	:195–197
enable_domain_heuristics	false	:198–201
cut_pool_max_size / cut_pool_max_age	2000 / 50	:206–207
solution_pool_size	10	:208
num_threads	-1（自动 → 硬件）	:213
cut_worker_queue_size / 超时	64 / 50ms	:214–215
max_plunge_depth	10	:216
share_conflict_learning_across_threads	false	:228
share_implications_across_threads（批量）	true（8）	:234
parallel_delay_until_incumbent	true	:243
parallel_serial_on_low_fractionality_root	true	:247
require_tree_exhaustion_certificate	true	:253–258
enable_reduced_cost_fixing	true	:270
enable_reduced_cost_proof_cut_resolve	true（ ≤ 4 个 LP）	:311
deterministic_parallel	false	:334
enable_lp_fallback（L1/L3 重试）	true（3 / 2）	:339–343
feasibility_pump_iters / max_dive_lps	5 / 60	:359–360
max_probe_vars / bound_propagation_rounds	50 / 3	:361–362
root_cut_stall_tol / root_cut_max_stalls	10^{-4} / 2	:367–368
root_cut_reject_nonmoving_rows（+ 最小提升量）	true（ 10^{-6} ）	:382–385
enable_cglp_cuts（候选数/效能/时间）	false（5 / 10^{-4} / 2s）	:503–513
probe_reliability / probe_max_candidates	2 / 3	:537–538
pool_cut_row_threshold / pool_cut_depth_limit	500 / 3	:550–551
stall_node_window	0（关闭）	:560–564
crossover_heuristic_freq / rins_frequency	100 / 200	:458–459
enable_lns（比例/节点数/时间/迭代数）	true（0.80 / 500 / 5s / 3）	:464–470
enable_work_stealing（双端队列容量）	true（256）	:482–483

参数	默认值	位置（未注明者均为 options.hpp）
random_seed	0x9E3779B97F4A7C15	:491
PresolveOptions.max_rounds	20	milp_presolve. hpp:53
PresolveOptions.max_probing_candidates / 深度	500 / 3	milp_presolve. hpp:64--65
PresolveOptions.zero_tol / bound_tol	10^{-10} / 10^{-9}	milp_presolve. hpp:70--71
BCDomain 可行容差 / 界 eps / 安全上限	10^{-7} / 10^{-9} / $8m$	bc_domain. hpp:51--52,194
kNodeFeasibilityTol / kIncumbentPruneTol	10^{-6} / 10^{-12}	bc_utils.hpp:70--73
kHybridBestBoundPeriod	10	bc_utils.hpp:1865
ConflictPool 容量 / 文字数 / 扫描上限	1024 / 64 / 512	bc_pools. hpp:266--267
CutFamilyStats 历史长度 / 最小效能	5 / 10^{-3}	bc_types. hpp:656--657

B 记号

- n : 变量数; $m = m_1 + m_2$: 约束数; nnz: 约束矩阵的非零元数; n_I : 整数变量数。
- N (或 Q): 开放节点数; P : 池规模 (割或子句); L : 子句长度; T : 工作线程数; k : 行长度。
- $\{v\} = v - \lfloor v \rfloor$: 小数部分。
- “文件: 行”形式的引用遵循 §1.3 的简写约定; 所有行号均指报告日期当日的工作树。

C 核实声明

各节在关键论断处标注 (V) 者, 均在报告撰写期间通过直接阅读所引用行完成核实 (包括: MIPModel/LPModel 的定义; api.cpp 的路由; branch_and_cut_lp 的跨度; Node 结构体与 branch_child(); 针对 search/、parallel/、root/ 模块、TreeDomainManager、age_and_purge 与 update_standard_form_bounds_incremental 的死代码 grep 检索; 核心 BCOptions 默认值; vendored 版 HiGHS 的逐节点求解与 cached_sparse_basis 重置; CutPool::add 去重循环; NodeQueue 存储与 pop_bestbound; 间隙早停守卫; flow-cover 排除注释; 隐含界割的代数推导; #if 0 的 MIR 注释; ConflictPool::propagate_impl; StandardFormLP 布局; BCDomain::restore/propagate; 分派器优先级列表; 预求解默认值; 十一处基持久化调用点)。§12 的普查计数是通过对工作树执行 §12.1 所述的成员访问与字面量 grep 得出的 (计数: 230 个选项字段 → 9 个零读取、71 个单读取; 370 个统计字段 → 19 个零引用、75 个单引用; bac.cpp 中 51 个不同的 getenv 字面量与 125 处 getenv (调用点; 零个 TODO/FIXME 标记; 9 个零读取选项声明已在所引用行逐一重读)。代码意图含糊的论断在陈述处标注 (U) (例如发现 4)。